Emergency Dispatch Priority and Coordination System

Final Project Report

ENSE707 Software Quality Assurance

Leo Cao (23211788), Fateh Bhular (23217987), Shawn Lee (23204035)

Auckland University of Technology

Table of Contents

1. Project Completion and Final Implementation..... 4

1.1 Project Overview..... 4

1.2 Final Scope and Changes Since Assessment 1 (What did we change?)..... 4

1.3 Final System Architecture and Design..... 6

1.4 Final Implemented Functionality..... 8

1.5 Final Prototype Evidence (Assignment 2 system)..... 10

1.6 Remaining Implementation Limitations..... 13

2. Requirements Verification and Traceability..... 14

2.1 Requirements Review and Changes Since Assessment 1..... 14

2.2 Final Requirements Traceability Matrix..... 16

2.3 Partially Met, Unverified and Out of scope Requirements..... 19

3. Test Execution and Final Test Results..... 20

3.1 Final Test Cases..... 20

3.2 Test Execution Results..... 22

3.3 Failed Tests and Corrective Actions..... 23

3.5 Remaining Testing Risks..... 25

4. Test Automation and Quality Engineering..... 26

4.1 Automation Selection..... 26

4.2 Automated Test Implementation..... 27

4.3 Automated Test Results..... 28

4.4 Automation Maintainability..... 28

4.5 Manual and Exploratory Testing..... 29

5. Continuous Integration and DevOps QA..... 31

5.1 Development and Integration Workflow..... 31

5.2 Continuous Integration Pipeline..... 32

5.3 Quality Gates..... 33

5.4 Smoke testing..... 34

5.5 CI Evidence and Limitations..... 35

6. Defect Management and Root Cause Analysis..... 37

6.1 Defect Management Framework..... 37

6.2 Final Defect Register..... 38

6.3 Root Cause Analysis of Significant Defects..... 39

6.3.1 D-01 (Restart Data Loss)..... 40

6.3.2 D-04 (Notification Failure Boundary)..... 41

6.3.3 D-05 (HTTP Test Database Isolation)..... 42

6.4 Defect Prevention and Quality Improvement..... 43

7. Quality Testing..... 44

7.1 Risk-Based Selection of Quality Areas..... 44

7.2 Quality Area (Security)..... 45

7.3 Quality Area (Performance)..... 48
7.4 Limitations..... 49
7.5 Recommended Improvements..... 51
8. Quality Metrics and Final Release Decision..... 52
8.1 Final Quality Metrics..... 52
8.2 Evaluation Against Requirements and Quality Attributes..... 53
8.3 Residual Risks and Release Limitations..... 54
8.4 Final Release Decision..... 55
9. AI-Assisted Development and Critical Reflection..... 56
9.1 AI-Assisted Activities..... 56
9.2 Representative GitHub Copilot Evidence..... 57
9.3 Human Review and Validation..... 59
9.4 AI-Related Quality Risks..... 60
9.5 Critical Reflection..... 61
10. Quality Improvement and Future Recommendations..... 61
10.1 Key Quality Lessons..... 61
10.2 Improvements to the Development and QA Process..... 63
10.3 Future Testing and Automation Improvements..... 63
10.4 Future Continuous Quality Engineering..... 64
11. Team Contribution and Professional Reflection..... 65

1. Project Completion and Final Implementation

1.1 Project Overview

There is a high volume of calls received by emergency dispatchers every day in New Zealand alone. Sometimes there are too many cases, so even office workers have to go out into the field to keep up with all of them *(1News Reporters, 2026).* This causes stress on the current emergency system, which underlines the need for a new response system that can effectively manage all emergency cases and coordinate response units for these cases. The Emergency Dispatch Priority and Coordination System was developed to improve the management of emergency calls, case handling, prioritisation, department routing, and response-unit coordination. The prototype from phase I allows the dispatcher to create a case and put it through the system, which assigns a priority. The case is then routed to the assigned departments, where they can view the case details on their dashboards with very quick updates. After a unit has finished a case, they can update it on their dashboard, and once all units have signed off, the case closes.



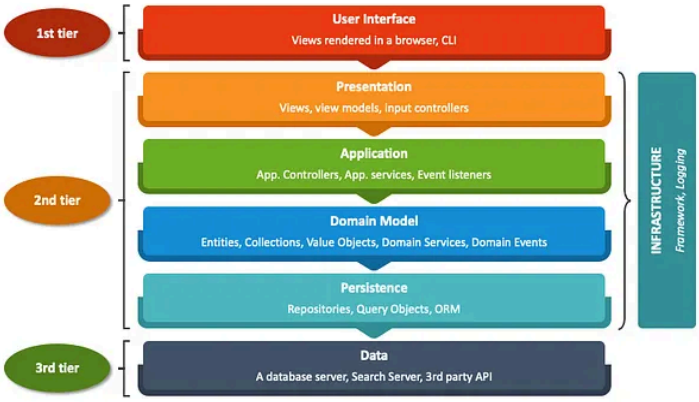
1.2 Final Scope and Changes Since Assessment 1 *(What did we change?)*

The original scope from assignment 1 was to develop an emergency dispatch priority and coordination system prototype that managed emergency cases, case prioritisation, case routing to departments, and response-unit coordination. The scope for the final assignment includes some changes to the features due to new findings, corrections required from the previous scope, as well as new ideas.

Scope Area	Assessment 1 Scope	Assignment 2 Scope	Justification
Case recording	Dispatchers can record caller details, incident type, description, location, severity, and required departments in a form. Submitting this form creates a new case.	**No scope change.** The required inputs remain validated, and the case number and timestamp are generated automatically during case creation.	Case recording provides the information that is used in prioritisation, routing, unit assignment, and testing. The automatic timestamp and case number improve consistency.
Prioritisation	The system automatically determines the case priority using the department-related rules. The dispatcher's selected severity is used when there are no keyword matches.	**Scope changed.** An authorised dispatcher can override the calculated priority by giving a mandatory reason, which is stored in the audit log.	Priority override allows a miscalculated priority to be corrected. The original priority, updated priority, user, reason, and the timestamp should all be recorded for accountability,
Routing	Cases are routed to every department selected by the dispatcher. Multi-department cases appear in all of the selected departments' interfaces.	**No scope change.** Selected departments can view the case on their dashboards. Cases are excluded from unrelated departments' dashboards.	This workflow satisfies the multi-department routing scope from assignment 1. There is no external emergency-service integration in this prototype.
Unit assignment and coordination	The first available unit from each required department will be assigned to the case. If no compatible units are available, the case will be in a queue.	**Scope changed.** Unit assignment assigns the nearest available unit using its stored location data. If no unit is available, the case remains in the queue.	Nearest-unit selection can reduce travel distance for the selected response unit and the expected time of arrival. This is supported by location data that is measurable, defined rules, and repeatable tests.
Case lifecycle, sign-off, and history	Cases use the following states: Open, In Progress, Closed. Each unit signs off individually, and the case only closes when all units have signed off. Dispatchers can search previous cases by caller name, case number, or date.	**No scope change.** The case lifecycle, unit sign-off, and case history are kept the same. Case closes after all required response units have signed off. History remains searchable by the inputs mentioned.	The lifecycle prevents cases from being closed too early on multi-department cases, and keeps all states consistent throughout the case lifecycle. The history supports traceability and access to previous cases.
Role-based GUI	There are separate interfaces and restrictions provided for dispatchers, departments, response units, and the prototype admin role. Department and response unit users are restricted to their assigned scope.	**No scope change.** The role-based interfaces and server-side access controls are kept. The admin is prototype only and is used for testing and demonstration.	Role-specific interfaces reduce the complexity and prevent users from accessing pages they shouldn't be allowed to access. Admin is kept for the prototype for easier testing and demonstration.
Architecture & Technology	This system uses C#, .NET, Razor Pages, HTML/CSS, MSTest, and GitHub with a modular design.	**No scope change.** The structure, repository abstractions, priority strategy, dependency injection, and central dispatch service are kept.	Keeping the existing technology stack avoids migration and regression risks. Explaining the architecture shows how the prototype supports maintainability, extensibility, and testing.
Security and Persistence	The system uses in-memory repositories and fake accounts in text files. Operational data was lost after restarting the program. Role-based authorisation was implemented, but not permanent storage or encryption.	**Scope changed.** Includes permanent database storage, password hashing, encryption at rest, and a persistent audit log. The audit log records important actions related to a case, timestamps, and the responsible user.	Persistent storage prevents important data from being lost after the program restarts. Password hashing and encryption reduce the risk of sensitive data being exposed if the database is accessed directly. Audit logging improves accountability, traceability, and security review.

1.3 Final System Architecture and Design

There are six layers used in our system architecture: Domain, Application, Logic, Infrastructure, Web, and Authentication. This separation in the system means that each part can focus on one responsibility, while still contributing a part to the complete dispatch workflow.



The **domain layer** contains the system's main objects, with each of them controlling its own rules, functionality, and state changes in the system. For example, assigning cases, releasing a unit, or closing a case. The main objects are cases, units, departments, priorities, and case states. The **application layer** is the coordinator of the system and tells parts of the system "what needs to happen". DispatchService is an example in the application layer, where it coordinates the case creation, prioritisation, routing, and case-related tasks. The **logic layer** contains the rules about the system objects that are independent of the interface and database. For example, KeywordSeverityPriority.cs implements the priority logic in the system, where its rules can be changed independently. The **infrastructure layer** contains all of the department, case, notification and other repositories. These have direct access to the database and its chosen operations. The **web layer** uses ASP.NET Core Razor Pages to display the interface and pages of the system. This includes the sign-in page, dispatcher dashboard, department dashboard, and all other system pages. Lastly, the **authentication layer** holds files that are responsible for credential checking and making sure users can only access information that is permitted by their role.

Our system implements **two main design patterns: the repository pattern and the strategy pattern**. The repository design pattern gives an abstraction layer between the business logic and data storage. This gives a consistent way of managing data while hiding the implementation details *(GeeksforGeeks, 2026)*. ICaseRepository, IDepartmentRepository and IDispatchNotifier all use the repository design pattern to separate the workflow from the storage. The strategy design pattern is a behavioural design pattern where a group of similar algorithms are used in a way that makes them interchangeable. This allows the algorithm to vary depending on what the client requires for their task, and enables these changes during runtime without modifying the existing code *(GeeksforGeeks, 2026)*. In our system, the strategy pattern was used for case priority calculation. The IPriorityStrategy interface defines the framework to calculate priority, and KeywordSeverityPriority provides the implementation of that framework.



We used **dependency injection** to connect the different layers and components of our system architecture. All of the repositories, priority strategy, notification service, and the dispatch service are loaded when the application starts. Rather than every class creating its own dependencies created unnecessary load on the system, the components are provided to those classes instead. This reduces the coupling between components while also making testing easier because the repositories and services can be provided for testing instead of the real implementations.

The DispatchService was implemented as the **central service for the main system workflow**. It coordinates the case creation, priority calculation, storage, unit assignment, notifications, unit sign-off, and the reassignment of released units to the cases in queue. The intention of this service was to prevent similar workflows from being implemented in different locations of the system. The pages receive user input and display information, while DispatchService coordinates what should happen to the case and units (backend of the application).

Domain-controlled state was used to reduce the possibility of invalid states during runtime. The states of cases and response units are controlled by the domain objects instead of the user interface. The

Case object controls whether the unit can be assigned, when a unit signs off, and whether the case should be Open, Closed, or In Progress. The Unit object controls its availability and the current case assignment. With domain objects controlling their own states, it prevents invalid states. For example, an unavailable unit still being assigned to another case, or a multi-department case closing when not all units have signed off.

All of the design decisions and patterns implemented improve the quality of the system and make it easier to test and maintain. The **layered architecture helps with testing**, due to all the business logic being separated from the web interface. This allows the business logic to be tested without needing to load the interface, allowing for easier automated testing. The **repository design pattern reduces the coupling** between components because the application layer doesn't worry about how the data is stored. Overall, the design decisions make the system more modular and easier to modify without needing to also modify other unrelated layers. The persistent database, password hashing, and audit logging added improve the reliability, security and traceability of the system. This is because it prevents data loss after the program restarts, protected information is secure, and important actions on cases are recorded for accountability.

1.4 Final Implemented Functionality

The following table summarises the core functionality implemented in the final version of the Emergency Dispatch Priority and Coordination System

Requirement	Final implemented functionality
FR-01 & FR-02	An emergency case is recorded, validated, and created based on caller, incident, location, severity, and department information is validated correctly
FR-03	Department users can only view their units and update unit location and their worker count. The unit availability is controlled by assignment and sign-off.
FR-04	The system calculates priority using department rules. An authorised dispatcher can override the priority result with a mandatory reason

FR-05	The cases appear on every selected department's dashboard and are excluded from unrelated department dashboards.
FR-06	The nearest available response unit is selected using its stored location data. If no unit is available, the case remains waiting until a compatible unit is free.
FR-07	Each department dashboard only displays its relevant cases and includes new routed cases automatically. The page refreshes every 5 seconds.
FR-08	The case follows the defined state rules (Open, In Progress, and Closed) in waiting, assignment, partial sign-off, and completion.
FR-09	Units become unavailable when assigned and available again after signing off. Two requests at the same time cannot give one unit two active assignments.
FR-10	Each unit that is assigned receives a notification containing the case number, incident type, and location. If the notification fails, it should not undo successful case creation or assignment.
FR-11	Units sign off individually; if units are still assigned to a multi-department case, its state should be In Progress. The case closes after all assigned cases have signed off.
FR-12	Dispatchers can search by caller name, case number, or date. Cases that match any of the fields are returned.
FR-13	Important actions include case creation, priority override, routing, assignment, sign-off, and closure. These are stored with the case action, user or system actor, and timestamp. Audit records should remain accessible after the program restarts.
NFR-S02 & NFR-U01	Dispatchers, departments, and response units can access only their allocated pages and scope. Any unauthorized page requests should be denied.
NFR-R03	Cases, units, assignments, notifications, accounts, and audit records should remain accessible after the application has restarted. Stored data should be retrieved without missing or changed fields.
NFR-S01	Sensitive account data should not be stored as readable plaintext in the local database. Authorised operations can encrypt, store, retrieve, and decrypt the data correctly.
NFR-S03	Account passwords should be stored using password hashing. Correct passwords should be accepted, while incorrect passwords and direct hash comparison attempts are rejected.
NFR-P01	Cases should be sent to the relevant department dashboards in under 2 seconds.

1.5 Final Prototype Evidence *(Assignment 2 system)*

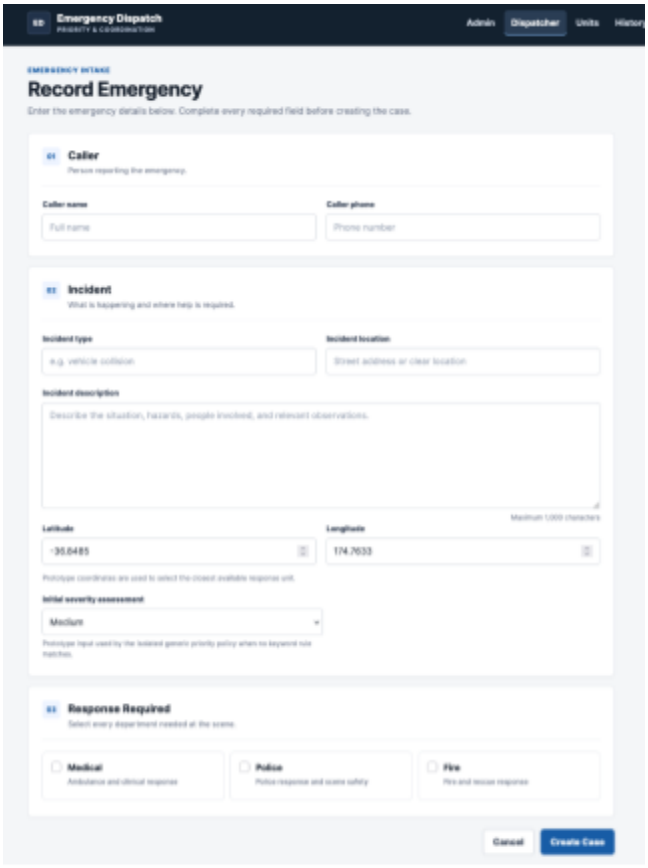


Figure 1.5.1: Dispatcher case-recording form

This is the /Cases/Create page where the caller details, incident details, location, reported severity, and department selection are captured. It provides dispatchers with a simple GUI to enter details rather than the older, confusing page. The location input is sent to the unit-assignment service, where it checks the coordinates of the location and finds the nearest available unit from each selected department. All mandatory information is checked using server-side validation, and the severity is checked using the priority system, which calculates the final priority for the case. The priority can also be overridden with a mandatory explanation of why it is changing. It will then be saved to the audit log, and the case details will be saved on the dispatcher's dashboard (*Figure 1.5.2*).

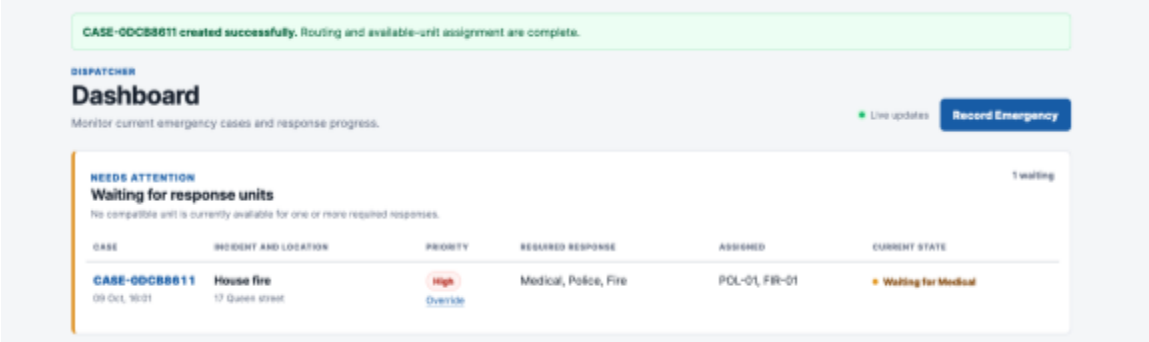


Figure 1.5.2: Case on dispatcher's dashboard

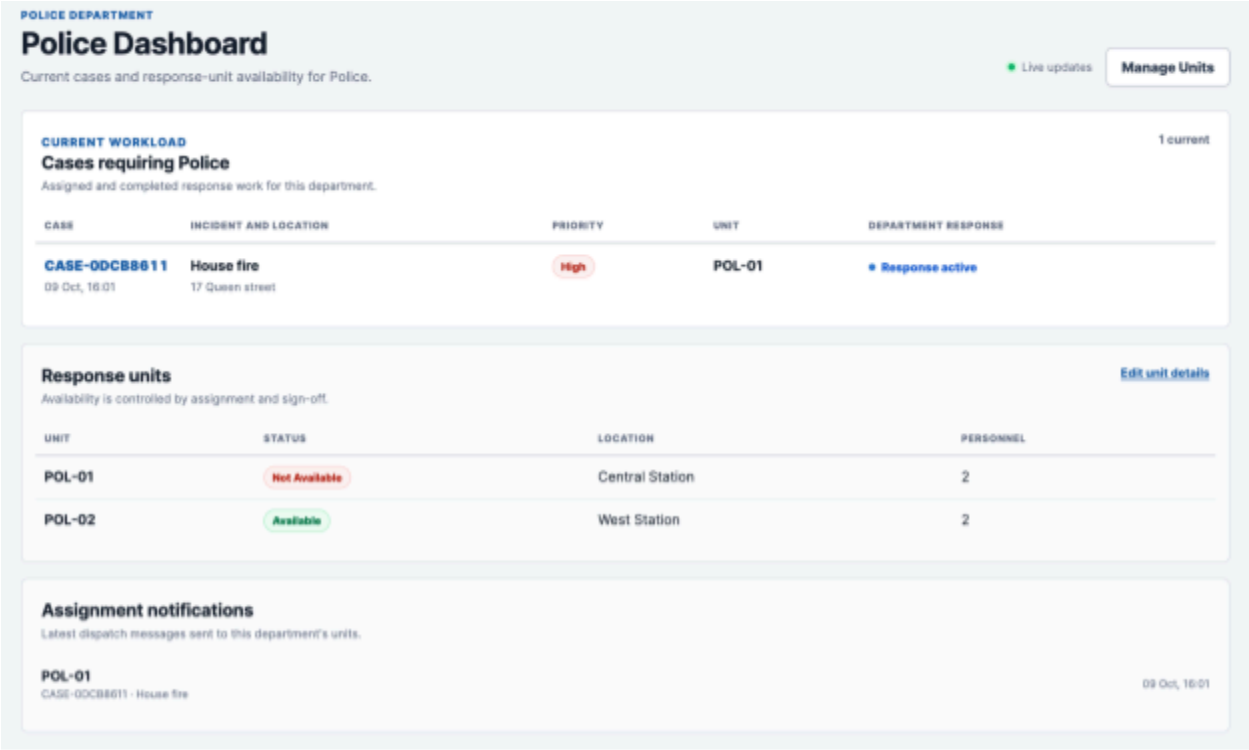


Figure 1.5.3: Department-specific dashboard

Figure 1.5.3 shows the police department's dashboard, where only cases assigned to this department will be displayed. Each dashboard shows the current cases that require police units, the list of all response units along with their availability status, and any notifications from the system. Each of these components is separated for easier viewing, with the goal of users being able to interpret critical information quickly. Dashboards reload every 5 seconds so that any new information is quickly updated, without the user having to manually reload the page, and reduce the chances of missed information.

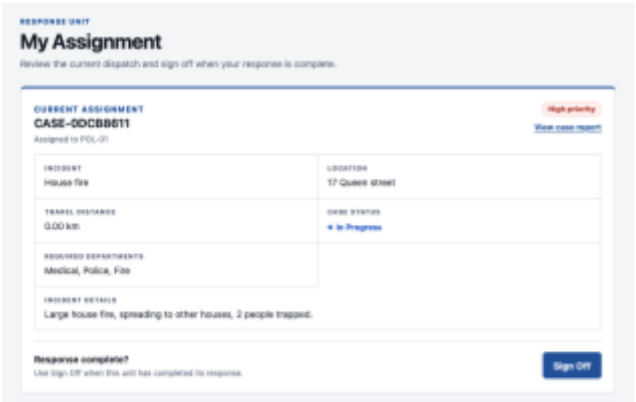


Figure 1.5.4: Response-unit workspace

Figure 1.5.4 shows the interface of the POLICE-1 unit, where they receive the current assignment and its details. The dispatcher's note is added because it provides important information that helps the unit prepare for the case. After the case is handled, the unit can sign off from the case, which only closes when all assigned units have signed off.

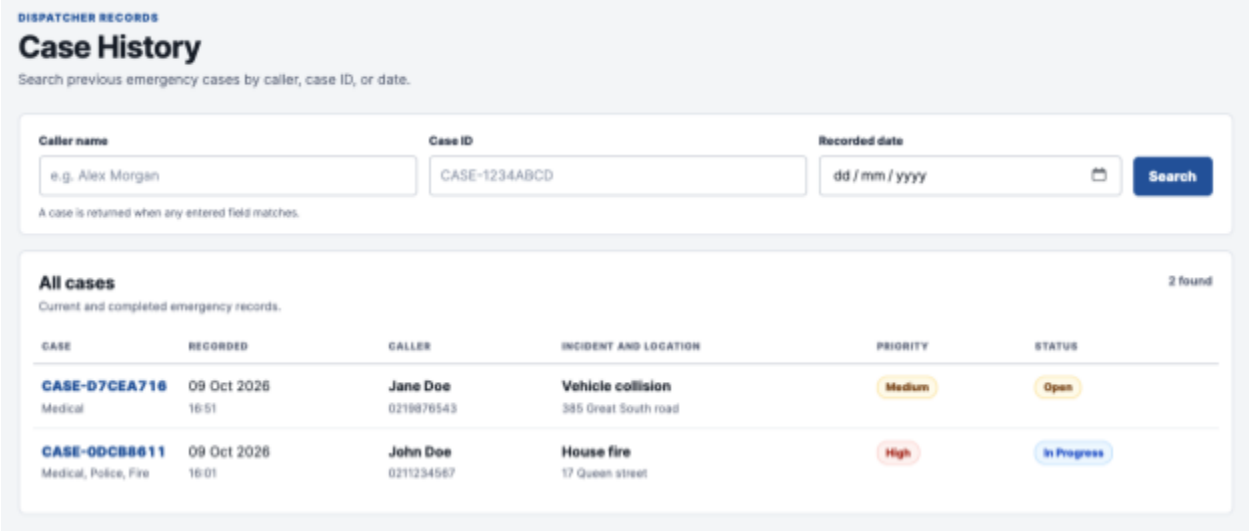


Figure 1.5.5: Case history showing case in queue

When no response units are available, the newly created case goes into a queue, with the status “Open”. Figure 1.5.5 shows the case that was just created with the status in the case history log. In the case history page, the dispatcher can search for cases by caller name, case ID, or the date it was created.



Figure 1.5.6: Audit log of case actions

Figure 1.5.6 shows the audit log of actions taken by different users. Every major action: creation, assignment, override, sign-off, and closure, is saved to this log to track who did what and to help with traceability. If an important event happened and someone had to figure out who performed a certain action, this log helps them find which user did it, making it easier for investigation if it occurs.

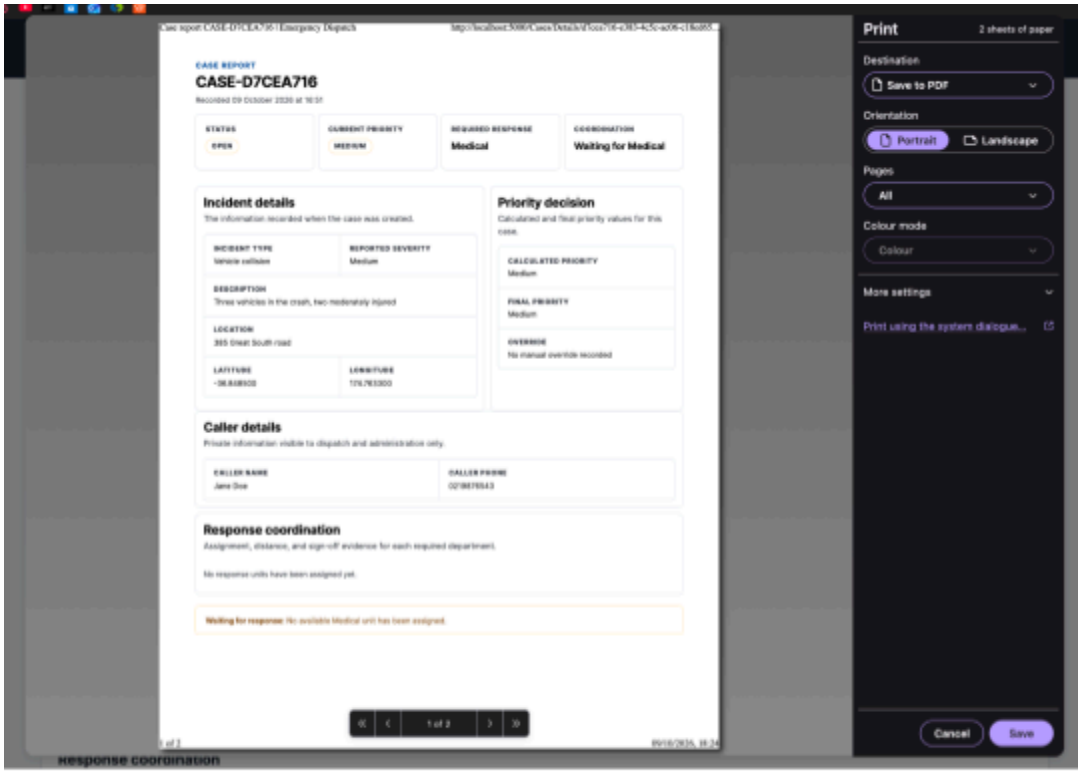


Figure 1.5.7: Printable case report

All cases have a “print report” button which generates a report of the case details and its full lifecycle. It shows the incident details, priority decision including the override details if any, caller details, the response coordination, and finally the case activity (displayed like the audit log, but only actions linking to this case).

1.6 Remaining Implementation Limitations

The final prototype implements all of the important requirements of a real-life emergency dispatch system, but it is still a local hosted prototype. These requirements include case recording, department routing, prioritisation, response-unit assignment, and audit logging. The routing is done locally, so it does not communicate with anything outside of this local hosted prototype, like real life emergency services. The distance calculation required coordinates, and adding a coordinate finder was out of the scope. Therefore, the dispatcher has to enter coordinates from a mock coordinate list, and use that for demos.

Dashboards are updated every five seconds using polling. In the future, we would like to add real-time push updates, which would allow departments to receive cases instantaneously, instead of five seconds later. Another limitation is that the SQLite database and locking have only been tested for this prototype and are not viable for large distributed systems.

As an Emergency Dispatch Priority and Coordination system, the security is not suitable for what it should be for a real-life emergency system. Our prototype still lacks secure database management, authentication methods, and firewalls for system protection, so while it is a functioning prototype, it should not be applied to real-life emergency scenarios or implemented into systems.

2. Requirements Verification and Traceability

2.1 Requirements Review and Changes Since Assessment 1

The refined functional and non functional requirements that were created in assignment one were revisited during the development and testing of the final prototype. The complete list of all requirements from assignment one is provided in **Appendix 5**.

The changes made in the final prototype showed that the original requirements no longer reflected the intended behaviour of the new prototype. As a result, we had to revisit the original functional requirements; making sure it properly described the intended behaviour, and adding new requirements for functionalities introduced in assignment 2. All these updated functional requirements for assignment two are provided in **Appendix 6**, and the upcoming section only mentions the changes that were made. Minor changes such as rewording or clarity were not included in the table below because they did not change the intended behaviour of the requirement.

The table only identifies requirements where the expected system behaviour or project scope was meaningfully changed, removed, or added during Assessment 2. Only the requirements which have been meaningfully changed in expected system behaviour or project scope are identified in the table.

2.1.1 Functional Requirement Changes

ID	Change from Assessment 1	Reason
FR-01	Added coordinates and reported severity, and clarified the required response departments and timestamp.	To support the updated case recording and priority/assignment requirements.
FR-03	Unit availability is now controlled automatically instead of being manually updated by departments.	To prevent inconsistent unit availability states.
FR-04	Added an authorised dispatcher priority override with a required reason.	To allow incorrect calculated priorities to be corrected with accountability.
FR-06	Changed unit assignment from the first available unit to the nearest available unit.	To make unit selection location-based rather than list-based.
FR-08	Removed the unused assigned-unit removal scenario and clarified the three case states.	To match the agreed case lifecycle.
FR-13	**new requirement;** added persistent audit logging for important system actions.	To improve traceability and accountability.

2.1.2 Non-Functional Requirement Changes

ID	Assessment 2 change	Reason
NFR-R03	Expanded persistence to cases, units, assignments, notifications, accounts and audit events across restarts.	To cover all persistent data used by the final system.
NFR-S01	Added AES-256-GCM encryption for selected sensitive stored data.	To protect sensitive data stored by the system.
NFR-S02	Added department/unit scope and denial of direct unauthorised requests.	To strengthen access control beyond the interface.
NFR-S03	**New requirement** for salted PBKDF2 password hashing.	To prevent passwords from being stored in plaintext.

2.2 Final Requirements Traceability Matrix

The final requirement traceability matrix links every requirement defined in the **Appendix 6** document to its implemented functionality, verification evidence, final result, and related defects where applicable. Section 3 gives more details on the testing procedures and execution results. *All the labeled defect IDs will be further elaborated in section 6.*

2.2.1 Functional Requirements Traceability Matrix:

ID	Final implementation	Verification Evidence	Result	Defect
FR-01	Validated Razor form records caller, incident, location, coordinates, severity and selected departments.	TC-01, TC-02 and HTTP workflow	**Passed**	D-06
FR-02	Creates one case with a generated identifier and timestamp. Initial status depends on unit availability.	TC-01, TC-03 and HTTP workflow	**Passed**	–
FR-03	Department users can view and update their units' location, coordinates and personnel count. Availability is controlled by assignment and sign-off.	FR-03 valid/invalid update tests, restart test and role tests	**Passed**	D-06
FR-04	Department-specific keyword rules calculate priority, with reported severity as the fallback. Authorised overrides require a reason.	TC-04 and PriorityOverrideTests	**Passed**	D-06
FR-05	Cases are visible to selected departments and excluded from unrelated department dashboards.	TC-03, TC-05, TC-09, role tests and HTTP tests	**Passed**	–
FR-06	Selects the nearest compatible available unit using Haversine distance. Cases wait when no compatible unit is available.	TC-06–TC-08 and DistanceAssignmentTests	**Passed**	D-03
FR-07	Department dashboards contain only relevant cases and reload every five seconds.	TC-09 and rendered-page HTTP refresh test	**Automated checks passed**	D-06

FR-08	Domain rules control Open, In Progress and Closed case states.	TC-03, TC-07, TC-10 and database lifecycle test	**Passed**	–
FR-09	Assignment makes units unavailable; sign-off releases them; locking prevents duplicate active assignment.	TC-06–TC-10, concurrency test and database lifecycle test	**Passed**	–
FR-10	Assigned units receive persistent case notifications. Notification failure does not undo a successful dispatch.	TC-03, notification-failure test and notification restart test	**Passed**	D-04
FR-11	Units sign off individually, and a case closes after the final required response signs off.	TC-10, database lifecycle test and HTTP workflow	**Passed**	–
FR-12	Dispatcher history supports OR-based search by caller, case number or date.	TC-11 and repository search tests	**Passed**	–
FR-13	Important case actions are stored persistently with actor, time, case and unit information.	AuditLoggingTests, PriorityOverrideTests and database lifecycle test	**Passed**	D-01, D-06

2.2.2 Non-Functional Requirements Traceability Matrix:

ID	Final implementation	Verification evidence	Result	Defect
NFR-P01	Database-backed multi-department dispatch must finish within two seconds locally.	TC-05; database-backed performance test	**Passed** ***(35 ms)***	–
NFR-P02	Support 100 concurrent users with responses within two seconds.	No representative concurrent-user load test	**Unverified, out of scope**	–
NFR-M01	Layered architecture and repository interfaces separate critical workflows from adapters; notification failure is isolated.	Architecture review; notification failure test	**Partially verified**	D-04

NFR-M02	Strategy and repository abstractions reduce changes to core workflow, but adding roles or department types still requires configuration and code changes.	Code and architecture review	**Partially met**	–
NFR-R01	Stored case information remains consistent across selected department projections.	TC-03, TC-05, TC-09; HTTP role tests	**Passed**	–
NFR-R02	Maintain 99.9% weekly availability.	No deployed monitoring period	**Unverified, out of scope**	–
NFR-R03	Cases, units, assignments, notifications, accounts and audit events persist across restarts.	SqlitePersistenceTests; multi-restart lifecycle	**Passed**	D-01
NFR-R04	Continue operating with 100 simultaneously open cases.	RequirementReliabilityTests.cs	**Passed**	–
NFR-S01	Selected sensitive database fields use AES-GCM; deployed TLS has not been independently verified.	Raw-database encryption and restart test	**Partially passed**	D-02
NFR-S02	Cookie authentication, middleware, role claims and scope checks restrict access.	TC-12; report access; real HTTP tests	**Passed**	D-06
NFR-S03	Passwords stored as salted PBKDF2 hashes; incorrect passwords rejected.	SecurityTests; restart authentication test	**Passed**	D-02
NFR-U01	Dispatcher, department, response-unit and prototype Admin interfaces are role-specific.	TC-12; page-model and HTTP tests	**Passed**	D-06
NFR-U02	Validation provides user-facing messages without exposing stack traces.	Invalid-input and login tests; manual review still required	**Partially verified**	D-06
NFR-U03	Users complete core tasks after no more than one hour of training.	No representative timed usability study	**Unverified**	–

2.3 Partially Met, Unverified and Out of scope Requirements

Most of the final requirements were fully verified, however, not all of them were. The following requirements were marked as not fully passed because some behavior was not completely verified, or their scope did not align within the final prototype.

Requirement	Status	Justification
FR-07	**Partially verified**	Passed automated tests for the dashboard and refresh, however the 5 second update still needs to be manually tested
NFR-P02	**Unverified**	Testing with 100 concurrent users was not done. This required load testing conditions that were outside the scope for our local prototype.
NFR-M01	**Partially verified**	Layered architecture separates critical workflows from adapters. and a regression test verifies notification-failure isolation.
NFR-M02	**Partially met**	The architecture supports extension, but adding roles or department types still requires configuration and code changes.
NFR-R02	**Unverified**	The prototype was not deployed and monitored for a week to ensure 99.9% availability.
NFR-S01	**Partially verified**	AES-GCM encryption at rest was tested, but TLS production configuration was not tested.
NFR-U02	**Partially verified**	The validation component was tested, but there was no complete manual review of the finally rendered error message output.
NFR-U03	**Unverified**	There was no user study showing that the main tasks could be carried out after only one hour of training.

3. Test Execution and Final Test Results

In the final prototype, we have added multiple changes to the prototype from the previous phase. These include priority override, nearest unit assignment, a database, improved security, and audit logging. These increased the scope of what the requirement would need to test; thus, the testing scope is increased to test these newly added features. The final testing stage is that all implementations would need to be verified on the updated functional and non-functional requirements in section 2. These testing approaches would include unit, integration, end-to-end, security and performance.

3.1 Final Test Cases

For the final test case, we have 167 tests which all pass; the table would consist primarily of core functional and non-functional requirements. These test cases cover different test types (integration, unit, and performance) and conditions (positive, negative, invalid input, boundary and error handling).

Test ID	Test Situation	Test Type	Condition	Status
TC-01	Creating emergency cases using valid incident information.	Integration	Positive	**Pass**
TC-02	Creating emergency cases using incomplete incident information.	Integration	Invalid Input	**Pass**
TC-03	Create, prioritise, store and assign cases to relevant departments.	Integration	Positive	**Pass**
TC-04	Calculate priority using department-specific keywords.	Unit	Positive	**Pass**
TC-05	Complete in-process multi-department within 2 seconds.	Performance	Positive	**Pass**
TC-06	Skip unavailable units and assign to the closest available and compatible unit.	Integration	Negative	**Pass**
TC-07	Queue the emergency case when there are no response units available and assign it after a unit is “open”.	Integration	Negative	**Pass**

TC-08	Prevent concurrent submission from assigning the same unit to two cases.	Integration	Negative	**Pass**
TC-09	Display only relevant department cases and include new cases after refreshing,	Integration	Positive	**Pass**
TC-10	Keep a case In Progress until the final unit assigned to it signs off.	Integration	Positive	**Pass**
TC-11	Search case history using caller name, case, identifier or date.	Unit	Positive	**Pass**
TC-12	Verify role based access and reject unauthorised access	Integration (Security)	Positive & Negative	**Pass**
TC-13	Reject priority override when dispatcher does not provide a reason.	Integration	Invalid Input	**Pass**
TC-14	Verify audit logs remain ordered by checking if the newest log appears first.	Integration	Positive	**Pass**
TC-15	Verify emergency case and unit assignment information is persisted when the database restarts.	Integration	Positive	**Pass**
TC-16	Verify the password hashing uses a different salt and rejects incorrect passwords.	Unit (security)	Positive & Negative	**Pass**
TC-17	Deny coordinate location outside of the permitted boundaries.	Unit	Boundary & Invalid Input	**Pass**
TC-18	Check whether notification failure does not interrupt case creation or assignment.	Integration	Error handling	**Pass**

These 18 test cases are demonstrations of the different testing approaches used throughout all the tests rather than the entire test coverage of the system. The detailed specification of these 18 test cases is provided in **Appendix 7**.

3.2 Test Execution Results

The final tests were executed against the completed system, covering 167 test executions mentioned in section 3.1. The results are summarised below:

Test Execution Results & Metrics	Number
Planned Test	167
Executed Test	167
Passed Test	167
Failed Test	0
Blocked Test	0
Not-Run Test	0
Pass Rate:	100%

The test pass rate was calculated from the total number of passed executed tests divided by the total number of executed tests times 100. Therefore giving 100% pass rate. This shows that the current system fully satisfies the expected outcome set for all tests. *(Figure 3.2.1 shows that all 167 tests had passed in Test Explorer in Visual Studio, tested in MSTest and used SQLite for database-related tests).*

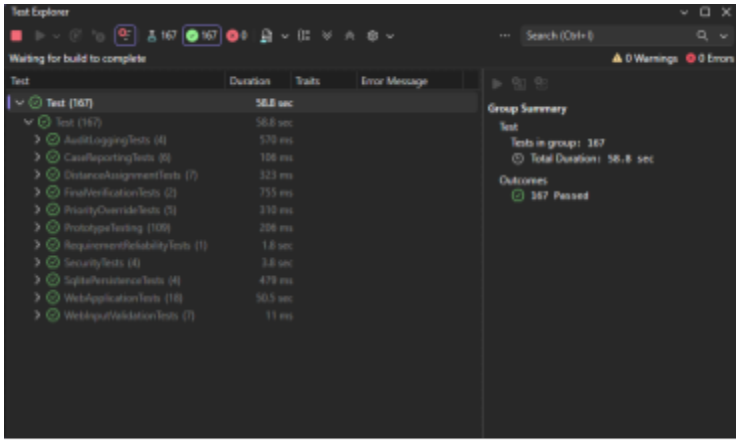


Figure 3.2.1 : All Test Case Pass Evidence

Although all 167 tests made had passed, this does not indicate that the system is fully verified for the system's full functionality and quality of the implementation defined by the requirements.

3.3 Failed Tests and Corrective Actions

For the initial test result ***(before section 3.1 Final Test Cases)***, there were 31 failed test cases out of 167 tests, resulting in a pass rate of 81.4%.

After reviewing all failed test cases, these test cases were mostly caused by the exception System.IO.IOException. These results occurred when trying to conduct testing while the cleanup process tried to delete the temporary SQLite database and locked up the files due to the connection still being used. These issues occurred throughout the different test cases such as WebApplicationTest, SqlitePersistenceTests, SecurityTests and AuditLoggingTests. Therefore, this is more related to the database connection rather than any failed testing logic implementation. These test cases would need to be fixed in order to verify that all functionalities are working as intended.

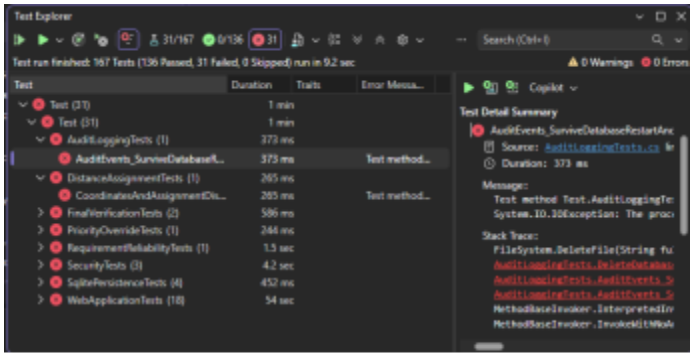


Figure 3.3.1 : System.IO.IOException

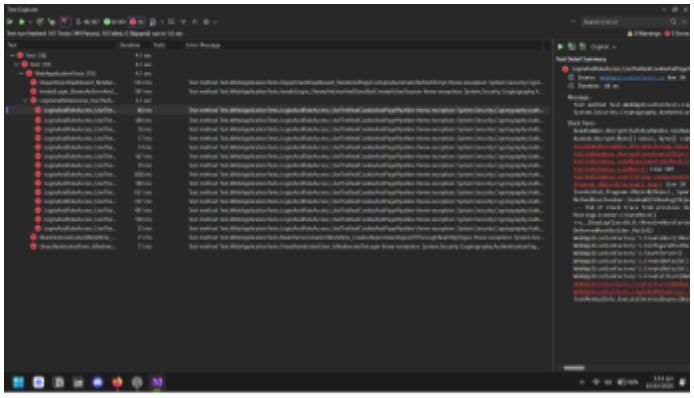


Figure 3.3.2: AES-GCM Decryption

The action taken to resolve this issue was to disable pooling for all the tests. Since each time the test requires an SQLite connection to the database, pooling could retain the database connection in the connection pool to be reused, thus preventing cleanup of the temporary database file created during the tests. Applying this change resolves the SystemIO.IOException issue.

After confirming that this update worked, this was then tested on a separate device to ensure that the issue is not related to the local environment. This resulted in 18 test cases failing rather than 31 on the initial device. These issues were related to *System.Security.Cryptography.Authentication TagMismatchException*. This occurred when the web application tried to decrypt stored data from the database.

Further investigation led to Program.cs, where this is not so much an authentication issue, but rather that the database is immediately initialised before the test could replace it with a temporary database. So the initialised database was changed so the database would only be initialised when the service requested it; this is implemented through utilising factory functions. This allowed the temporary database to be used instead of unnecessarily initialising the original database setup.

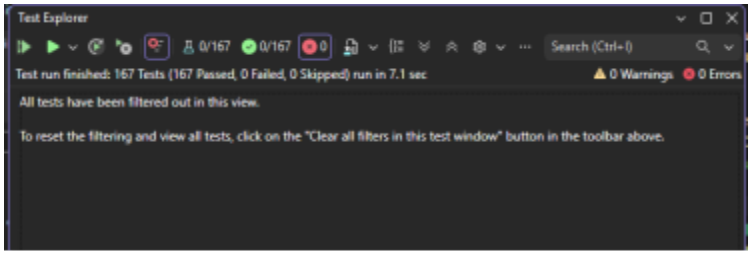


Figure 3.3.3: Result after the changes

Applying both changes towards the final testing implementation. All 167 tests passed with 0 failed tests

3.3.1 Regression testing

After applying the test changes to the initial testing described in section 3.3 *(disabling pooling and implementing a factory function in database initialisation)*, regression testing was performed by reusing the tests applied before the updated changes. The changes could affect data persistence, case creation, authentication, and case assignment, which are critical requirements; therefore, all 167 tests were rerun instead of only the previously failed tests, as the changes made could have a wide blast radius.

After rerunning the 167 tests, the previous 31 tests that had issues with database handling were resolved, as the other tests had remained as pass. This indicates that, after the modification, no other new bugs had been found after changes made in order to solve the initial test issues.

3.5 Remaining Testing Risks

Although the 167 tests did cover major areas in the system, there are still areas where the current test still has not been verified / fully verified. These requirements that are not yet verified mainly consist of non function requirements where the current testing does not provide enough evidence. Below are critical requirements that have not been checked as well as possible future testing:

Risk	Remaining Risk / Impact	Possible further testing
NFR-P02 *(Concurrency user performance)*	The current concurrency test accounts for concurrent competing case submission but does not test for 100 simultaneous users.	Since this requirement is stated in section 2.2 that it is out of scope for this project, a test would not be created for this project, but would need to be tested when the system is further developed to be in production.
NFR-R02 *(System availability)*	This could cause system wide downtime which is important that the system is not down as the system deals with emergencies.	This is another testing that is simply out of scope for this prototype but for production, monitoring of the system over a long period of time would be required to ensure it reached 99.9% availability.
NFR-S01 *(Data transfer security)*	The test currently verifies for stored data at rest but not when the data is being transmitted.	This test is important especially when reaching production. This could be tested on HTTP / TLS and verifies that the communication is encrypted between the user and the system.
NFR-U03 *(Usability)*	Dispatcher learning curve could be slower leading to slower case assignment thus impacting units ability to quickly respond.	The test could be conducted with a group of representative dispatchers to measure the time of completion between common tasks in the system. Feedback could be received for the system usability as well.

4. Test Automation and Quality Engineering

We implemented automated tests on selected behaviour where the inputs were clear and had deterministic outputs. The priority of automated tests was given to the high-risk features that were used frequently, and to those that were vulnerable to errors during regression testing. Automated testing is important in this project because we need to keep checking the core functionality after code changes and before changes are accepted: priority calculation, unit assignment, lifecycle transitions, persistence, access control and input validation. Because these have deterministic outputs, automation supports software quality assurance by providing fast feedback and reduced manual regression effort (Dr Siddique, 2026).

4.1 Automation Selection

The areas mentioned above were selected for automated testing because their behaviours were essential to the project and were the core functionality. The following test cases received automated testing because failure in any of these areas would cause our system to fail, and they need to be tested after code changes and before changes are accepted.

Automated area	Reason for implemented automation	Evidence
Case creation and Priority	This is a core workflow that has deterministic outcomes, along with different keyword combinations	PrototypeTesting, TC-01 to TC-04
Unit assignment and waiting queue	An incorrect assignment could allocate an unavailable unit to the case or fail to queue the case.	DistanceAssignmentTests, TC-06 to TC-08
Case and unit lifecycle	The transitions between states need to be consistent from case assignment to sign-off.	TC-07, TC-10, database lifecycle tests
Database persistence	The automated tests simulate the database restarts and verify that the information is saved when the database is reopened. Easier than manual verification	SqlitePersistenceTests, FinalVerificationTests
Security and access control	Positive and negative tests should be repeated after changes are made to authentication, roles, or pages.	SecurityTests, TC-12 WebApplicationTests

Priority override and auditing	Overriding needs validation, authorisation, and all details. This has important components that should be checked repeatedly.	PriorityOverrideTests, AuditLogging Tests
Invalid and boundary input	It is important to check valid, invalid and boundary values quickly.	WebInputValidationTests, DistanceAssignmentTests
Error handling	Confirms that a notification failure will not stop the case creation or assignment from being successful.	Appendix02_NotificationFailure_DoesNotBreakCriticalDispatchWorkflow test
Reliability and performance	Need to check dispatch speed and whether the system can handle 100 open cases. Automation makes it easier for the developer, instead of having to manually check the performance.	DatabaseBackedMultiDepartmentDispatch_CompletesWithinTwoSeconds test, RequirementReliabilityTests
HTTP Web workflow	This is how a user uses the system, so it's important to always check login, access control, case creation, and case sign-off using the actual web pages.	WebApplicationTests

4.2 Automated Test Implementation

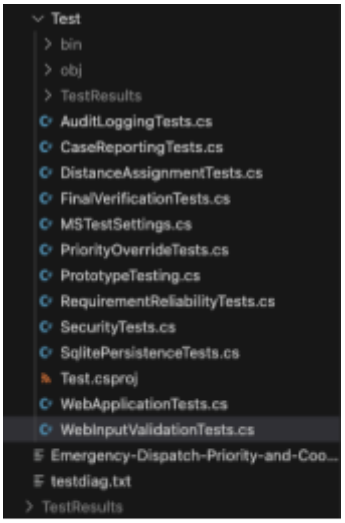


Figure 4.2.1 : Testing Directory Structure



Figure 4.2.2: AAA testing

All automated testing was implemented using MSTest 4.0.2, which was written in C# using .NET 10. We used MSTest because it integrated our project solution easily and required little effort during setup. As shown in Figure 4.2.1, the automated tests were separated by the functionality or quality area that was being tested. For example, DistanceAssignmentTests.cs contains code for testing the assignment of response units using their distance, and SqlitePersistenceTests.cs contains code for testing overall database persistence.

All of the tests used AAA testing as shown in Figure 4.2.2. Arrange creates the case, unit, repository, etc. Act performs the action/behaviour that is being tested. Assert compares the actual result with the expected result. Unit testing was used to check the individual components and how they worked in isolation. This included the priority keyword calculation, coordinate validation, and input boundary. Unit tests are fast because they only test small pieces of logic instead of using the whole web application.

4.3 Automated Test Results

show the automated run result, probably 167/167 passed, with Test Explorer/log screenshot evidence. Don't repeat all of Section 3; just focus on the automation result.

```
Test summary: total: 167, failed: 0, succeeded: 167, skipped: 0, duration: 2.4s
Build succeeded in 3.0s
fatehbhular@Fatehs-MacBook-Pro Emergency-Dispatch-Priority-and-Coordination-System %
```

Figure 4.3.1 : Automated Testing Results

The final testing suite produced 167 automated MSTest executions where all 167 tests passed, including all data-driven rows *(Figure 4.3.1)*. There were no failed, blocked, or skipped tests because there was an acceptable amount that could be run in a short period of time. These tests include unit, integration, database, security, reliability, performance, and HTTP behaviour. While this result shows that all of our expected outcomes were satisfied, it doesn't mean that our system is defect free. Because not everything was tested, as shown in section 3.5, defects relating to performance, usability, and accessibility could still remain in the system.

4.4 Automation Maintainability

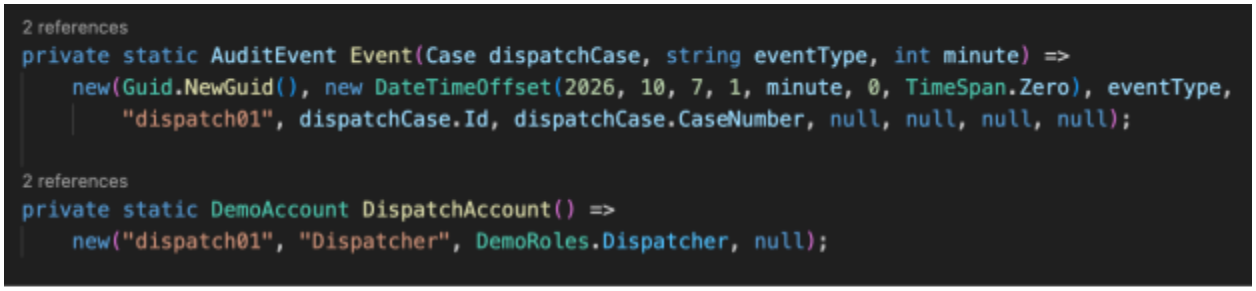
When creating the tests, we sorted them into different files for easier maintenance and organisation. Figure 4.2.1 shows how there were tests for different behaviours of the system like audit logging, priority override, web application, etc. All tests used the Arrange, Act, and Assert template, shown in figure 4.4.1, which is commonly used to write well defined tests in code. The test names describe the input or situation and what the expected result of the test is. This makes it easier for any developers looking through the test files, and helps with understanding what tests are being performed, making the test suite maintainable.

```
public sealed class CaseReportingTests
{
    [TestMethod]
    0 references
    public void DispatcherReport_LoadsCaseAndOrdersItsAuditTimeline()
    {
        // Arrange
        var cases = new InMemoryCaseRepository();
        var audit = new InMemoryAuditRepository();
        var dispatchCase = AssignedMedicalCase();
        cases.Add(dispatchCase);
        audit.Add(Event(dispatchCase, AuditEventTypes.UnitAssigned, 2));
        audit.Add(Event(dispatchCase, AuditEventTypes.CaseCreated, 1));
        var page = CreatePage(cases, audit, DispatchAccount());

        // Act
        var result = page.OnGet(dispatchCase.Id);

        // Assert
        Assert.IsInstanceOfType<PageResult>(result);
        Assert.AreSame(dispatchCase, page.DispatchCase);
        Assert.IsTrue(page.CanViewPrivateDetails);
        CollectionAssert.AreEqual(
            new[] { AuditEventTypes.CaseCreated, AuditEventTypes.UnitAssigned },
            page.AuditEvents.Select(item => item.EventType).ToArray());
    }
```

Figure 4.4.1 : Arrange, Act, Assert structure



```
2 references
private static AuditEvent Event(Case dispatchCase, string eventType, int minute) =>
    new(Guid.NewGuid(), new DateTimeOffset(2026, 10, 7, 1, minute, 0, TimeSpan.Zero), eventType,
        "dispatch01", dispatchCase.Id, dispatchCase.CaseNumber, null, null, null, null);

2 references
private static DemoAccount DispatchAccount() =>
    new("dispatch01", "Dispatcher", DemoRoles.Dispatcher, null);
```

Figure 4.4.2 : Helper methods for testing

For one section of tests, some tests would use identical setup steps or functions. To make the test suite more maintainable, helper methods were implemented to create services, databases, and input objects. This not only reduced the total number of code, but also made it easier to create new tests that need to use the helper method.



```
6 references
private static (SqliteCaseRepository Cases, SqliteDepartmentRepository Departments) CreateStore(string path)
{
    var database = new SqliteDatabase($"Data Source={path};Pooling=False");
    return (new SqliteCaseRepository(database), new SqliteDepartmentRepository(database));
}
```

Figure 4.4.2 : Helper methods for testing

Database tests were separated from the rest of the tests for test isolation. Each test was given their own temporary SQLite database and files, so that they wouldn't be added to the database used for normal demonstration. After each test was executed, the database along with the files were deleted.

4.5 Manual and Exploratory Testing

Automated testing reduces the work needed for repeated regression testing. While it can consistently check functionality after code is pushed, it has limitations and cannot confirm that the entire system works as intended. This is where manual and exploratory testing is used for specific behaviours. For components like app styling, error messages, or visuals, manual human validation is required, as written tests cannot confirm it is to our expectations.

Manual Area	Why manual testing is needed
Visible dashboard refresh	An automated test already verifies that the 5-second refresh happens, but a person should confirm this with their own eyes that the information appears.
Validation and errors	Automated tests check the validation rules, but a person should determine if the error messages are readable and understandable.
Visual style of the web app	Automated testing cannot determine if the spacing, colours and alignment of components are as they should be. For this, humans are needed to check if the styling meets the requirements.
Printable case report	The printing case preview should be checked by a human to make sure its readability, layout, and information is correct, and nothing is cut out, or broken.
Responsive layouts	The layout of the web app should be tested manually by actively resizing the window, the human can then determine if the layout of the page is usable with different sizes.
Exploratory testing	A human tester should try out different sequences of actions that are not written in any tests or scenarios. This can help reveal any defects or workflows that were not predicted in the system.
Role-based access	There are several automated tests that verify the user-based restrictions, but manual testing should be used to test if the user can see the expected tools and layout on the interface.

The areas above will be testing using manual and exploratory testing, because it requires visual validation from humans. During manual testing, all evidence can be recorded using screenshots along with the date and name of the tester for accountability. This evidence should also include the comparison between the expected and actual behaviour. This will make the evidence of manual testing be as informative as automated tests.

5. Continuous Integration and DevOps QA

5.1 Development and Integration Workflow

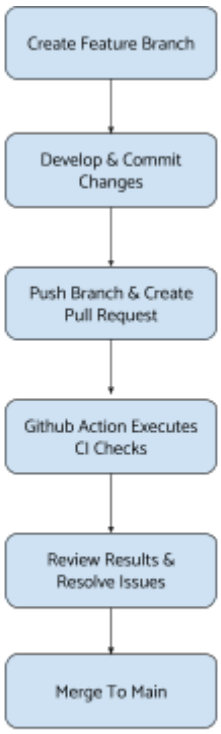


Figure 5.1.1: Development and Integration Workflow

Throughout the development workflow, github version control was used for this system. Each functionality is developed through different branches outside of the main branch. Commits were made to each individual branch when the feature was successfully developed in the branch. The separate branches allows each developer to work independently without impacting the main branch though conflict would still occur during integration to the main branch. Then a pull request is created where to review the changes made and merge conflicts are checked to ensure that there are no conflicting codes and resolve any issues found.

Continuous Integration workflow is included by the use of Github Actions which allows the automatic workflow to be applied whenever there are changes pushed towards the testing branch or pull request with the main branch. In this automatic workflow, the system is built and automatic tests are run to make sure the behavior of the systems passed the set tests. If the test or build failed developers would review the changes before allowing the merging with the main branch.

5.2 Continuous Integration Pipeline

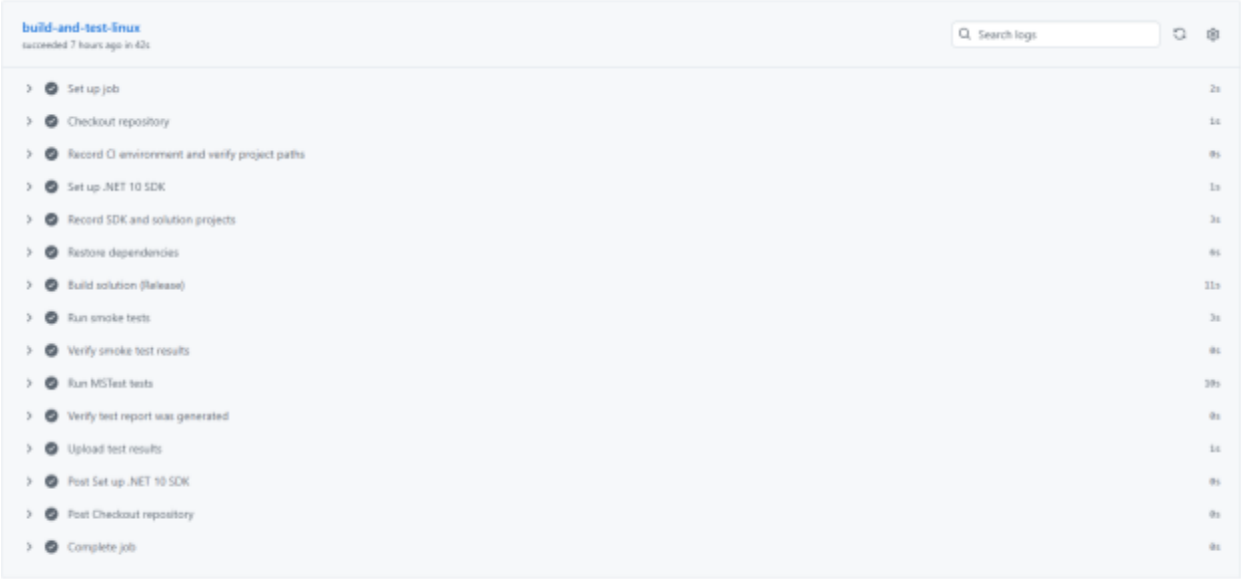


Figure 5.2.1: Successful Github Action CI Pipeline Execution

The continuous pipeline was configured in the file .github/workflows/ci.yml using Ubuntu linux runner with the .NET SDK. The pipeline is triggered when there is a pull request to the main branch on updates and creations or towards the Testing Branch for CI. Once this is triggered the pipeline goes through a series of steps. It first ensures that the path and environment are correct inside the repository. This is followed by dependencies installing and building. This checks whether the emergency dispatch system complies properly and successfully.

After confirming the build is successful, The smoke test was utilized (The core functionality tests would be stated in section 5.4) to check the core functionality of the system. If the smoke test passes, the rest of the automated test would run. If all tests are passed, the test results are stored in a TRX file for reviewing and checking failures if there is.

5.3 Quality Gates

Quality Gates are used to stop code from other branches from being incorporated into the main branch without meeting certain criteria, using automated checkpoints to ensure that each point the updated code reaches meets a set standard. Implementing quality gates improves the Quality test in many ways, including having a baseline quality check and detecting defects earlier. However, this would still need human intervention to check commits that failed to meet the checkpoint standard. Below are the set criteria that the produced code needs to meet before reaching integration:

Quality Gate	Passing Criteria	Consequences	Justification
Environment Check	The runner is using linux, and the solution has to be .slnx and the test.csproj file exists.	CI fails, restore dependency, verify build, smoke test, full automated test and produce test report are skipped.	It's important before the other checks, since it validates that the project is properly configured.
Restore Dependency	Check that the project can restore dependencies before building.	CI fails, verify build, smoke test, full automated test, and produce test report are skipped.	Ensures that the dependencies required to build and test are available.
Verify Build	The system build correctly compiles.	CI fails, smoke test, full automated test, and produce test report are skipped.	Make sure that the system compiles without errors, this allows it to be detected earlier to prevent unnecessary test execution.
Smoke Test	All 89 core functionality tests passed.	CI fails, full automated tests, and produce test reports are skipped.	This is to make sure that the main emergency dispatch functionality fully passes the test and that it acts as a regression test to ensure that at the very least the main functionality of the system is operational and unchanged.

Full Automated Test	All 167 automated testing passed.	CI fails, report is skipped.	A form of regression test to check if any introduced problems affected the existing functionality covered by the automated tests.
Produce Test Report	The full test report is produced and saved.	CI fails, because the test file is missing or empty.	The report is used for developers to analyze changes and review issues.

These quality gates are arranged sequentially from the layout of the table, meaning that the failure of a quality gate means the quality gates below would be skipped. The additional CI enforcement has been added to Github branch protection to ensure that CI must pass before a pull request can be merged into the main branch.

5.4 Smoke testing

Test Case	Core Functionality	Purpose
TC-01	Create an emergency case using valid information	Check that the emergency cases can be recorded successfully.
TC-03	Route a case to the correct department	Verify that the department receives the correct case.
TC-04	Calculate the emergency case priority	Verify the case incident receives the appropriate priority.
TC-06	Assign an available response unit	Check that available and compatible units are assigned properly.
TC-10	Complete a case after unit sign-off	Check that units were able to sign off and the corresponding case is “Closed”.

The smoke tests were selected from the identification of core functionalities in section 1.4, where these tests represent the main emergency dispatch workflow, TC-01, TC-03, TC-04, TC-06 and TC-10. Any failures of these functionalities would interrupt the main dispatch process.

There are 89 recorded tests in the smoke tests stage, although there are 5 core functional test cases that were selected, this is because TC-04 which uses DynamidData to execute 85 different input combinations to check that the priority calculation rules all pass. The other 4 test cases each have 1 test which makes 89 tests in total. In the successful CI run in the testing branch, 89 executions were passed with 0 failures.



Figure 5.4.1: Example of Smoke Test Quality Gate Results

5.5 CI Evidence and Limitations

Figure 5.5.1 shows an example of all 167 automated tests passing in GitHub Actions. It ensures that modifications to the code do not affect functionality that was tested in the automated tests by rerunning all the automated tests. Figure 5.5.2 shows an example of the report being generated and uploaded as an artifact in GitHub Actions, so that issues could be investigated through analyzing the produced test report. Branch protection has been implemented to enforce the required CI checks before merging into the main branch, preventing pull requests from being merged without passing the implemented CI.



Figure 5.5.1: Example of Total Automated Tests in Test Quality Gate Results



Figure 5.5.2: Example of Test Report Created & Uploaded

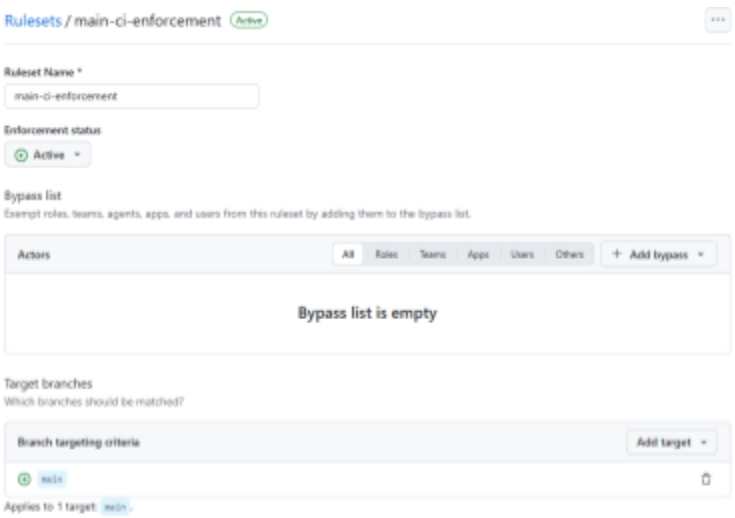


Figure 5.5.3: Enforcement ruleset Evidence

The implemented quality gates are an improvement to the development process. The automatic pipeline helps check whether the code builds properly, check dependencies, perform testing, and generate test reports are important. However, further quality gate implementation in the pipeline would be required to improve the quality of the code that is merged into the main branch. The pipeline lacks a measurable way of code coverage. Even though there are 167 automated tests, it does not mean that most of the systems have been tested. Additionally, there is no vulnerability checking of the system; there are security checks, but there are no security checks for dependencies, etc.

6. Defect Management and Root Cause Analysis

6.1 Defect Management Framework

Defects and quality issues were identified throughout development, code review, and testing. We documented major defects in the defect register and assigned them a severity, which is their impact on the application, and also a priority, which is how quickly it should be resolved. Every time a defect is detected and confirmed, its root cause is analysed, the appropriate action is taken, and then finally a regression test is done before closing the defect.

Below is the table that defines our severity and priority categories;

Category	Level	Definition
Severity	Critical	Prevents safe use, results in loss of significant data, or results in unauthorised access to significant data
Severity	High	Breaks a major system function
Severity	Medium	Impacts a non-critical workflow or reduces the quality of a verification criterion
Severity	Low	Minor issue, or graphical issue
Priority	P1	Needs to be fixed immediately
Priority	P2	Must be fixed before final release
Priority	P3	Future improvement
Priority	P4	Optional improvement

The defect statuses used were New, Confirmed, In Progress, Fixed, Retested, Closed, and Deferred.

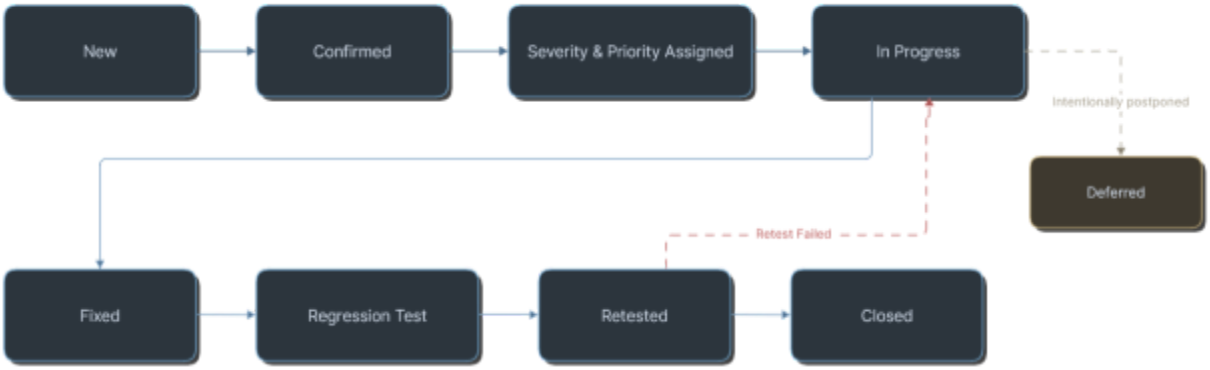


Figure 6.1.1 : The defect lifecycle used in the project

The status of each defect was tracked throughout its lifecycle, as shown in Figure 6.1. Once a defect is detected and confirmed, it is assigned a severity and priority before the defect has been analysed and the correct solution has been implemented. After a solution is proposed, a regression test is done, and then the defect is retested once again. If the retest is successful, the status of the defect will change to 'Closed', and if the retest fails, it is returned to 'Progress' for further analysis. For defects held in Progress', if they are intentionally postponed, they can be moved to 'Deferred'.

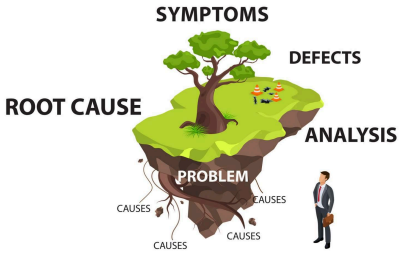
6.2 Final Defect Register

The final defect register contains the major defects and quality issues that were identified during the development of the final prototype. All the defects found were corrected, retested, and closed before the final prototype was completed.

ID	Defect / Quality Gap	Severity / Priority	Fix	Status
D-01	Data was lost after restarting the application.	High / P1	Switched from in memory storage to SQLite.	Closed
D-02	Credentials and sensitive data could be readable in storage.	High / P1	Added PBKDF2 password hashing and AES-GCM encryption.	Closed
D-03	The first available unit was selected instead of the nearest unit.	High / P2	Added coordinate distance assignment	Closed
D-04	Notification failure could cause a successful dispatch to appear to fail	High / P1	Isolated notification failure from the critical dispatch workflow.	Closed
D-05	HTTP tests could use the developer database.	Medium / P1	Replaced with an isolated temporary test database.	Closed
D-06	Earlier tests lacked negative and real workflow coverage.	Medium / P2	Added more negative, boundary, restart, audit, and HTTP testing	Closed

The root causes, corrective actions, regression evidence and preventive measures for the selected significant defects are analysed in Section 6.3.

6.3 Root Cause Analysis of Significant Defects



The following three defects were selected for further root cause analysis, these being D-01, D-04 and D-05. These defects as shown from table in section 6.2 are related to system reliability, fault handling and testing quality. The five whys technique was used because each of the defects had a relatively clear sequence of events that we can follow in order to find the root cause of the high level problem. In this case the fishbone diagram was not suitable for this cause because the defects were not complex enough to require an analysis across a large number of possible causes. In this case, the five whys technique was much more straightforward to determine the root cause, and the possible preventive actions.

6.3.1 D-01 *(Restart Data Loss)*

Problem: Cases, assignments and unit changes were lost after the application restarts

Impact: The application is not able to reliably preserve important emergency case information

5 Why's

1. ***Why was data lost?*** Because the app stored data only in memory.
2. ***Why did the app only store data in memory?*** Because the original prototype used in memory repositories.
3. ***Why was this still a problem in the final version?*** Because the final system needed data to persist after a restart, but the storage design had not originally supported that.
4. ***Why was this not found earlier?*** Because the previous tests reused repository objects rather than testing an actual restart with a durable database.
5. ***Why did the previous tests do that?*** Because they were built from the original in-memory prototype rather than the restart recovery.

Root cause: Original in-memory storage design was carried on from the prototype, and earlier tests also were built according to this design, so this was not properly tested.

Corrective Action: Replace in-memory repositories with the SQLite-backed ones so that data is persistent after a restart.

Preventive Action: Add restart testing to make sure stored data is not lost when the application is closed and reopened.

Verification: SqlitePersistenceTests and FullDatabaseLifecycle_RemainsCorrectAcrossMultipleRestarts verify that the stored data remains available after reopening the database.

6.3.2 D-04 *(Notification Failure Boundary)*

Problem: A notification failure could cause a successful dispatch to look like it had failed.

Impact: Even though it indicated an error, the case and unit assignment could have already been saved.

5 Why's

1. ***Why did the dispatch appear to fail?*** Because the notification service threw an error back through the dispatch process.
2. ***Why was the error allowed to pass through the dispatch process?*** Because there was no failure boundary around the notification service.
3. ***Why was there no failure boundary?*** Because notifications were handled as part of the same workflow as case storage and unit assignment.
4. ***Why were they handled as part of the same workflow?*** Because the design did not separate critical operations from non-critical operations.
5. ***Why were critical and non-critical operations not separated?*** Because their different failure behaviour had not been considered in the original workflow design.

Root cause: The workflow did not separate non-critical notification failures from critical dispatch operations

Corrective Action: Notification failures were isolated so they could no longer cause an already successful dispatch to appear unsuccessful.

Preventive Action: Separate critical and non-critical operations during design and include failure tests for non-critical components.

Verification: Appendix02_NotificationFailure_DoesNotBreakCriticalDispatchWorkflow uses a failing notifier and verifies that the case is still stored and assigned.

6.3.3 D-05 *(HTTP Test Database Isolation)*

Problem: HTTP tests are able to use the developer database instead of their own isolated test database.

Impact: Tests could be reading and writing development data, making the test results more unreliable.

5 Why's

1. ***Why could the HTTP tests be using the developer database?*** The test server was still using the application's Sqlite Database.
2. ***Why was it still using the application's database?*** We changed the test connection setting, but this didn't replace the existing database service.
3. ***Why didn't changing the connection setting replace it?*** The application had already registered the Sqlite Database before the service even started.
4. ***Why was the registered database not replaced?*** Because the test setup assumed changing the configuration was enough to use the temporary database.
5. ***Why was this problem not identified sooner?*** The HTTP tests mostly read data, so there weren't any clear signs we were using the wrong database.

Root cause: Our test setup changed the database configuration but didn't replace the existing SqliteDatabase service.

Corrective Action: HTTP test setup removed the existing database service and replaced it with a temporary database for the test workflow

Preventive Action: HTTP and other integration tests use a separate temporary database which can't access development data.

Verification: WebApplicationTests now replaces SqliteDatabase with a temporary database used only for the test workflow.

6.4 Defect Prevention and Quality Improvement

The defects that were identified during the project were able to help us implement actions that would help prevent the same problems from occurring again.

Issue Identified	Preventive Measure	Quality Improvement
Data was not preserved after system restart	Added restart and persistence regression tests	Improved reliability
Sensitive stored data was easily readable to the public	Added security tests for password hashing and stored encrypted data.	Improved security
The wrong unit could be selected	Added nearest unit, unavailable unit & tie case tests	Improved assignment accuracy
The dispatcher could be affected by notification failures	Separated critical and non critical operations, and added test failure scenarios.	Improved fault tolerance
The HTTP tests were able to access the development data	Used separate temporary databases for HTTP and integration tests	Improved test reliability

In general, the select analysts were able to help improve the quality of the product, as well as the overall quality assurance process. The product quality was improved through stronger persistence, security, assignment behaviour and fault handling. And the quality of the QA process was through implementation of better regression testing, failure injection testing, test isolation and negative & boundary testing. All these measures contribute in helping to prevent similar defects from recurring rather than a temporary solution to a defect.

7. Quality Testing

7.1 Risk-Based Selection of Quality Areas

Testing is prioritised based on risk based selection, which is identifying the area in the system that is the most critical quality risk. In the emergency dispatch system, the table below are the quality areas in the system that need to be considered for quality testing. The rating is based on **appendix 8** where it explains how the risk rating criteria is determined.

Quality Area	Related Requirements	Likelihood	Impact	Overall Risk	Testing Response
Security	NFR-S01, NFR-S02, NFR-S03	Medium (2)	High (3)	High (6)	Tests were focused on verifying authentication, role base access control and data encryption.
Performance	NFR-P01, NFR-P02	Medium (2)	High (3)	High (6)	Measure the system against the 2 second processing time from NFR-P01.
Reliability	NFR-R01, NFR-R02, NFR-R03, NFR-R04	Medium (2)	High (3)	High (6)	Integration and database testing to verify data persistence, cases information consistency and unit assignment.
Usability	NFR-U01, NFR-U02 NFR-U03	Medium (2)	High (3)	High (6)	Input validation and role specific interface were verified however, representative user test has not occurred.
Accessibility	No requirement specified	Medium (2)	Medium (2)	Medium (4)	No testing has been implemented to test accessibility in any form.

Maintainability	NFR-M01, NFR-M02	Medium (2)	Medium (2)	Medium (4)	Automated regression tests and smoke tests were applied through the CI pipeline to help detect defects introduced by code modification.
Compatibility	No requirement specified	Medium (2)	Medium (2)	Medium (4)	No testing has been implemented to test compatibility in any form.

Based on this risk assessment table, Security and performance were selected for further quality evaluation. Although Reliability and Usability also shared a similar overall risk score, due to the importance of both security and performance these are first evaluated for improvements. Security is a large focus as protecting the sensitive information shared to the dispatcher during case creation as well as preventing unauthorized access are needed where information about the dispatcher is constantly saved and audited. Performance needs to be heavily tested upon as case creation and case assigned to the right corresponding unit are important for the units to reach the incident at a critical moment. Both of these have existing automated tests and measurable requirements that can be evaluated on this current prototype design.

7.2 Quality Area (Security)

The security testing was created to verify the defined requirement based in section 2.2, these cover NFR-S01, NFR-S02 and NFR-S03. The automated tests were focused on verifying sensitive data encryption, authentication, role base access control and hashing. These were tested using MSTest through unit test and integration to check whether it behaves to the defined expected result.

Through the security quality area, there are 11 unique security tests and 35 individual test cases when data driven tests were included. In this evaluation of the security test, only a select few representative tests are discussed in detail.

Requirement	Test Method	Testing Approach	Expected Behavior
NFR-S01	SensitiveCaseFields_AreEncryptedInSQLiteAndDecryptAfterRestart	Integration	Select the sensitive case fields encrypt them to be stored, while the correct information can be retrieved after reopening the database.
NFR-S02	LoginAndRoleAccess_UseTheRealCookieAndPagePipeline	Integration	Authorised users can access their permitted interfaces, while unauthorized users are unable to access them.
NFR-S02	OperationalReport_IsLimitedToSelectedDepartmentsAndAssignedUnits	Component	Only authorised departments and assigned units can access the report page of an existing emergency case, while unauthorised users are denied access.
NFR-S03	PasswordHasher_UsesDifferentSaltAndVerifiesOnlyTheCorrectPassword	Unit	Identical password generated different salt hashes. Correct passwords are accepted while incorrect are rejected.
NFR-S03	Database_StoresPasswordHashInsteadOfPlaintextPassword	Integration	The passwords stored in the database are correctly hashed instead of storing plain text.

The NFR-S01 integration test passed, with 1 test executed and 0 failures. The actual result matched the expected behaviour defined in the testing table.

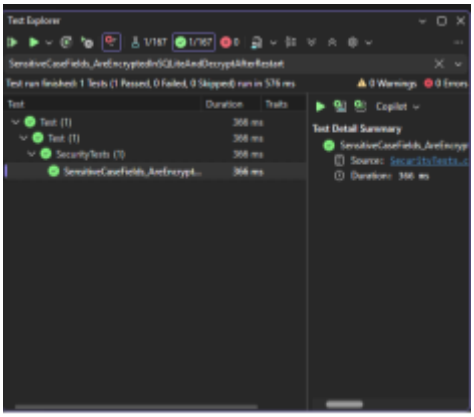


Figure 7.2.1: NFR-S01 Security Test Execution Result

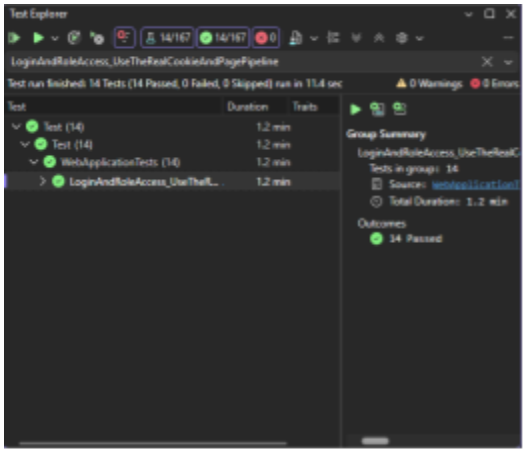


Figure 7.2.2 : NFR-S02 Authentication and Role-Based Access Control Test Results

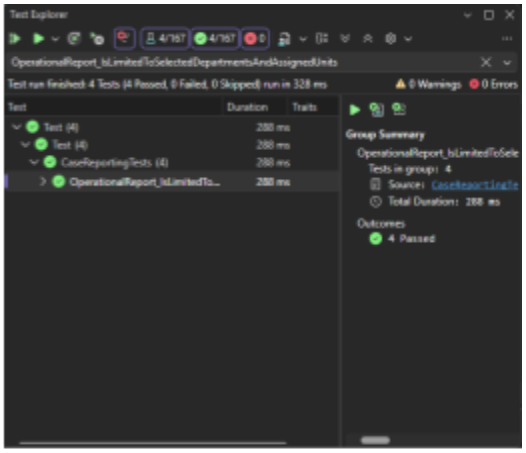


Figure 7.2.3 : NFR-S02 Case Report Access Restriction Test Results

The two selected NFR-S02 test methods completed successfully, with all 18 test cases passing and no failures recorded.

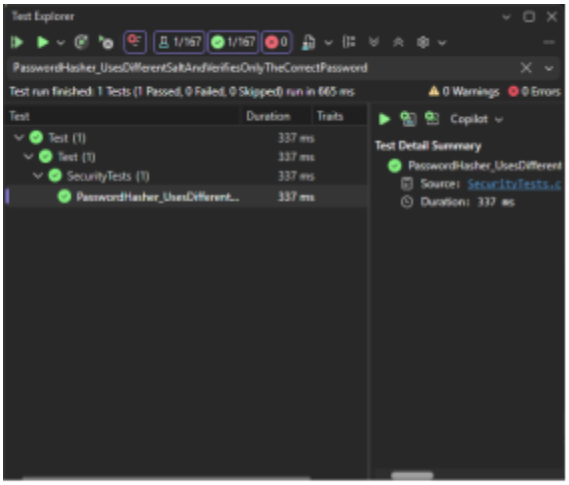


Figure 7.2.4 : NFR-S03 Password Hash Check Result

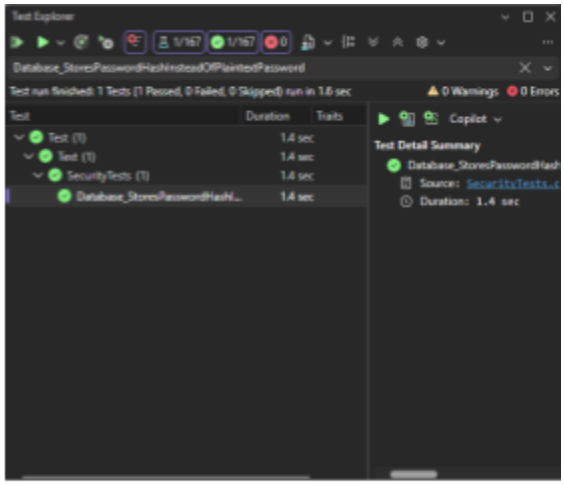


Figure 7.2.5 : NFR-S03 Store Hashed Password Result

Both NFR-S03 security tests were executed using MSTest, these 2 tests passed and no failures recorded. The results matched the expected behaviour for password hashing, verification and secure password storage.

7.3 Quality Area (Performance)

The performance quality area was tested with 2 requirements defined in section 2.2; these are NFR-P01 and NFR-P02. The two automated performance test methods are two individual test cases. These test cases were focused on measuring the time required to create and dispatch an emergency case to its respective departments using SQLite and in memory repositories. Both tests use a stopwatch to measure the dispatch operation and compare the results to the 2-second limit defined in requirement NFR-P01. NFR-PO2 was not verified as this requirement was out of scope, as discussed in section 2.3. NFR-P02 was not included because the current prototype has not been tested with 100 concurrent users thus this remains unverified.

Requirement	**Test Method**	**Testing Approach**	**Expected Behavior**
NFR-P01	DatabaseBackedMultiDepartmentDispatch_CompletesWithinTwoSeconds	Integration	The system completes a dispatch with three departments in less than 2 seconds, with unit assignment, notification and audit events all recorded correctly.
NFR-P01	TC05_InProcessMultiDepartmentSubmission_CompletesWithinTwoSecondsWithoutDataLoss	Integration	The system completes a dispatch with three departments in less than 2 seconds without losing case information or unit assignments.

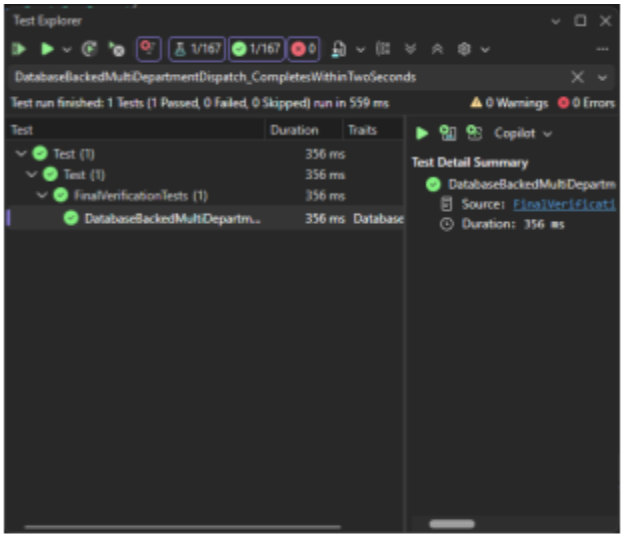


Figure 7.3.1 : NFR-P01 SQLite Stored Multi-Department Dispatch Performance Test Result

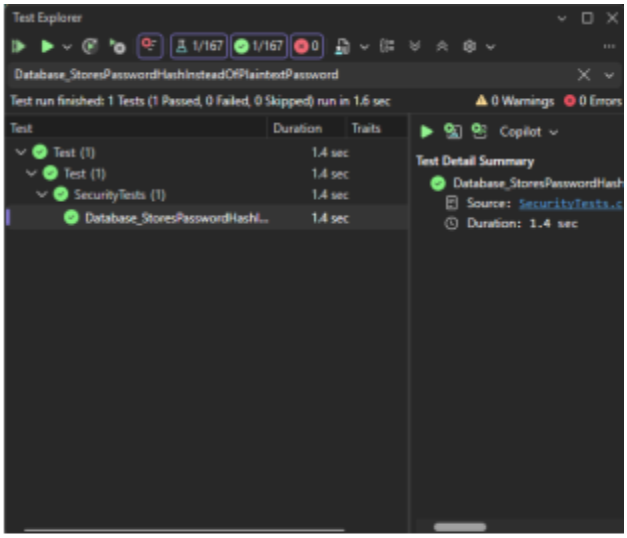


Figure 7.3.2 : NFR-P01 In-Memory Multi-Department Dispatch Performance Test Result

Both NFR-P01 performance tests showed 2 tests passing and no failures recorded in Test Explorer. The results indicate that the measured multi-department dispatch operations completed within the required two-second threshold under the tested conditions both in-memory and SQLite database. However the Test Explorer execution time is the overall duration not the exact dispatch processing time.

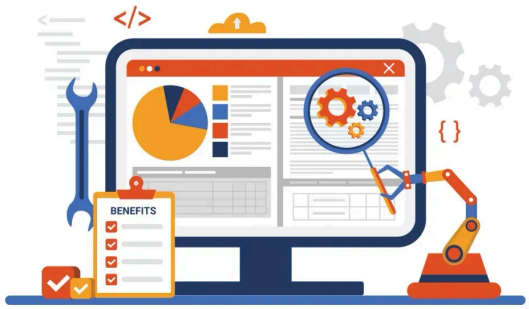
7.4 Limitations



For NFR-S01, the test only inspects the raw database values of the caller name and latitude to check for the expected encryption prefix that starts with "ENC1: ". The test also did perform retrieving the caller's phone number, case description, and location correctly after reopening the database, but this only confirms that the original values can be restored. Therefore, this does not confirm that all the field values were stored in their encrypted format. Another limitation already described in section 3.5 is the TLS encryption, where transmitted data over a network is not tested. This limitation means that NFR-S01 is not fully verified.

NFR-S02 tests did not cover every possible combination of user roles, permissions, and secured operations when testing for access restriction. Though there are some tests that verify access to restricted existing cases, other HTTP integration scenarios only check for nonexistent record cases. The test as well does not demonstrate that unauthorized attempts are not able to modify case information.

In NFR-S03, the test verified that different salts produce different hashes for identical passwords and passwords are correctly stored as hashed in the database. However, testing does not cover invalid input and error-handling situations. An example is the database storing corrupted hashes or salts. Additionally, the test does not verify the configuration of how the passwords are hashed; the algorithm used is PBKDF2-SHA256, and the iteration count is the amount of processing the algorithm performs, making password guessing more computationally difficult. Therefore, since the test did not take into account whether the configs had been changed, a reduction in iterations could weaken password security.



The current limitation for NFR-P01, is that it tests for an individual dispatch operation but does not measure whether multiple emergency cases can be processed simultaneously. Furthermore, the performance test only measures the time taken for the dispatch service to process a case. There is no measurement for when the dispatcher submits a case to when it is displayed on the interface. This means that the test does not measure the complete response time experienced by the dispatcher.

7.5 Recommended Improvements

The NFR-S01 requirement should expand its test cases to include all possible sensitive data stored in the SQLite database. Additional testing should be implemented to test corrupted encrypted data and incorrect encryption keys, as well as implementing TLS testing.

NFR-S02 should introduce more coverage in access control for all the different roles (department, units, dispatcher). This should include tests to ensure emergency cases are not modified without authorisation, and this would improve the testing beyond just testing assigned permissions. The NFR-S03 could be improved by adding testing for unexpected data or corrupted data stored in the database and include tests for checking the configuration of the hashing algorithm.



For the performance requirements, NFR-P01 should include implementing a response time test from when the dispatcher submits the case to when the case is displayed on a web interface for the dispatcher. Expanding on the response time test, it should also consider processing 2 or more emergency cases simultaneously and measuring the response time. This is to verify that the operation's performance remains acceptable within realistic situations.

To meet the NFR-P02 requirements, load testing would need to be implemented to test 100 concurrent users. Although this requirement was outside the scope of the current prototype, future development of this system would require this test implementation to verify the system can maintain response time for 100 concurrent users within the 2-second threshold. This provides more evidence when assessing the system readiness for deployment.

8. Quality Metrics and Final Release Decision

8.1 Final Quality Metrics

The final quality metrics bring together the testing, requirements, CI, and defect-management evidence from Sections 2-7. These metrics show the quality achieved under the tested conditions and identify the areas where verification is not complete.

Metric	Final result	Interpretation
Automated test pass rate	167/167 executions passed with zero fails, blockages, or unexecuted tests	All expected results during execution were met. The data driven rows are included *(Sections 3.2 and 4.3)*.
Regression result	167/167 executions all passed after changes	The entire suite was rerun and there were no failures with tests *(Section 3.3.1)*
CI and smoke test results	Successfully built; 89/89 smoke executions and 167/167 full-suite executions passed	The core workflows and full automated suite passed in the reported GitHub actions execution *(Sections 5.4-5.5)*.
Requirements recorded as passed	19/27 *(70.4%)* passed, 12/13 functional and 7/14 non functional requirements	Most functional requirements passed, while verification of non-functional requirements was less complete *(Sections 2.2-2.3)*
Remaining requirement statuses	4 partially verified, 1 partially met, and 3 unverified	The two unverified requirements, NFR-P02 and NFR-R02, were outside of the scope of testing for the prototype *(Section 2.3)*.
Registered defect severity and status	6 closed defects, 4 high and 2 medium severity. There are 0 recorded open defects	All issues in the final defect register were corrected and retested. However there could still be undetected ones *(Section 6.2)*
Local dispatch performance	2/2 performance tests passed the two second threshold	The tested in-memory and SQLite dispatch operations finished within two seconds. However the performance with multiple users and dashboard update speed were not tested *(Sections 7.3–7.4)*.

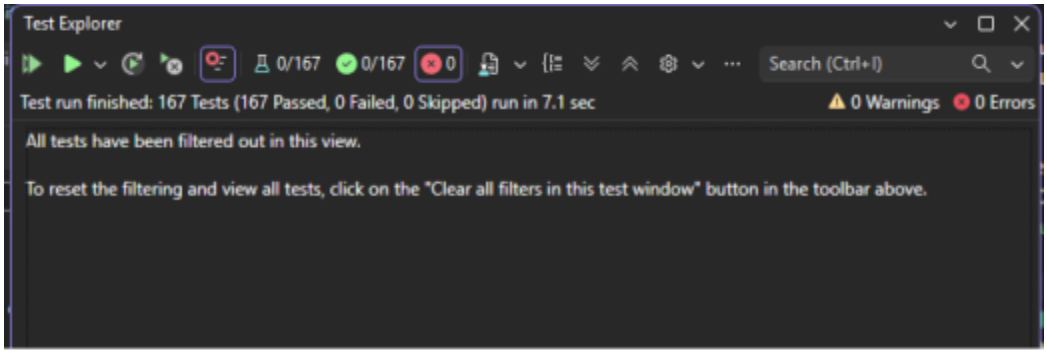


Figure 3.3.3: Result after the changes

All the 167 test cases executions were passed, giving us a 100% pass rate. However, only 19 of the 27 requirements were recorded as passed. The others were partially verified, partially met, or unverified, as explained in Section 2.3. This is the number of requirements that were recorded as passing, it does not reflect the code coverage or confirm that every acceptance criteria was met.

From these results, they give us confidence in the programs tested functionalities and fixes, however they do not do the same for the entire system. They do not prove that the entire system is fully verified. Furthermore, the code coverage was not tracked, and the smoke, regression, and security tests overlap with the complete suite so where should not contribute with the total of 167.

8.2 Evaluation Against Requirements and Quality Attributes

All the testing results show that the main dispatch flow works for the areas of case recording, prioritisation, department routing, nearest-unit assignment, queuing, and sign off. The tests verify that all the behaviours works as intended, such as incomplete submissions are rejected, priority overrides require a reason, unavailable units are excluded from assignment, and cases remain In Progress until the final required unit signs off. These results show that the requirements and acceptance behaviours have been met, however they do not confirm that every acceptance criteria is met *(Sections 2.2 and 3.1).*

The reliability tests showed that the data remained consistent between departments, and more importantly; no data was lost when the application restarted. The system was also able to pass reliability tests where 100 cases were open, but this does not mean that the system can support 100 concurrent users or 99.9% uptime. The performance testing was also able to show that the dispatching process was able to be processed within 2 seconds under the local testing conditions. *(Sections 2.2 and 7.3-7.4).*

The system was also able to pass the security tests on access restrictions, password hashing and the encryption checks for the selected stored fields. However NFR-S01 is only partially verified as encryption testing was limited to the selected fields and TLS was not checked. The role specific interface was successfully tested, but visible dashboard refresh, error message clarity, and the one hour training target were not fully verified *(Sections 2.3 and 7.4)*.

The implementation of the layered architecture along with the repository interfaces adds abstraction and supports maintainability. However its relevant requirement NFR-M01 is still only partially verified because the complete separation of the architectural responsibilities have not been fully tested. NFR-M02 is partially satisfied because adding new roles or department types still requires the developer to make some changes to the code.

8.3 Residual Risks and Release Limitations

Each of the six recorded defects has been corrected, retested and closed. However, the limitations and incomplete testing described in Sections 1.6, 3.5 and 7.4 still leave the following risks.

Area	Remaining limitation	Release Implication
Performance and availability	Performance with 100 concurrent users and 99.9% weekly availability are not verified.	The performance testing was all done using the local prototype, so this does not guarantee the product will work with production workloads.
Security	Security TLS was not verified and not all sensitive stored fields and access control scenarios were tested.	The current security checks do not provide enough evidence that it can safely handle real emergency information.
Dashboard updates and usability	Dashboards use five second polling. Visible refresh, error message clarity, and training time were not fully verified.	Users may experience delays or difficulties that the automated tests did not detect. This worsens the overall user experience.
Deployment and integration	SQLite and locking were tested only locally. Coordinates are entered manually, and routing does not connect to real emergency services	The prototype cannot be treated as an operational emergency dispatch system.
Accessibility and compatibility	Neither area was tested.	Usability across different user needs, browsers, and devices is not known.

These gaps are not recorded as open defects in the final register. They limit what can be concluded from the passing tests and must be considered when deciding the release scope.

8.4 Final Release Decision

The final decision is that the system is **not ready for operational release as an emergency dispatch system.** All the 167 automated tests were executed successfully, the CI build and smoke tests also ran successfully, and all the six identified defects were closed. These outcomes provide assurance that the main dispatch workflow operates as intended, such as case creation, prioritisation, routing, unit assignment, persistence and sign off.

However, the system is still not ready for operational release because the performance with 100 concurrent users, 99.9% weekly availability, TLS security, and representative usability remained unverified. The performance tests are especially an area of concern because a real dispatch system would require 99.9% uptime, and also suitability for 100 concurrent users. Any compromise in these areas could result in the risk of real lives. Furthermore, dispatch systems handle real sensitive data from patents which our system does not handle properly from end to end. These shortcomings led us to conclude with this decision.

The version we have currently built is still acceptable for prototype demonstration and further development. But real deployment would require the remaining shortcomings to be addressed properly, verifying the outstanding requirements, and testing in an accurate deployment environment.

9. AI-Assisted Development and Critical Reflection

9.1 AI-Assisted Activities

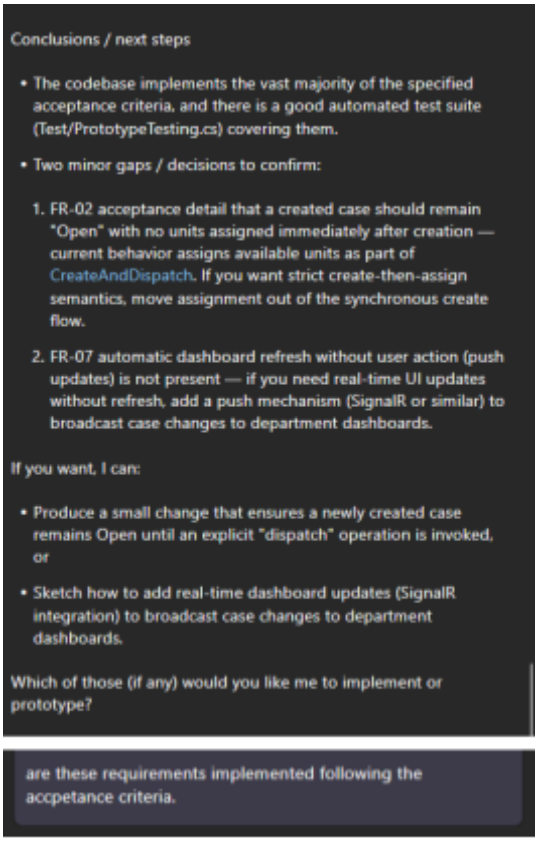


Figure 1.2.1: Github Copilot Evaluation

In the previous assignment, the functional requirements and non-functional requirements are defined and the acceptance criteria are created to describe expected behaviour of the core functional requirements. GitHub Copilot has been used to review the requirements that are gaps identified of core functionality with acceptance criteria. FR-02 and FR-07 are flagged as gaps that need to be considered for further changes. FR-01, FR-04, FR-05, FR-06, FR-08, FR-09 and FR-11 are not found as issues and are considered potential issues by GitHub Copilot.

It identified that FR-02 acceptance criteria is not fully met, this criteria in FR-02 "Given a case is newly created from a recorded call, when creation completes, then the case status is set to "Open" with no units assigned to it yet". FR-07 is flagged because GitHub described that the dashboard does not automatically refresh without user action, which doesn't meet this: "Given a department's dashboard is open, when the information of a case changes e.g. priority, location, then the dashboard should refresh and show the new changes automatically." and "Given a department's dashboard is open, when a case is routed to that department, then the dashboard should display the new case automatically."

When personally investigating whether the FR-02 implementation meets the acceptance criteria gaps during testing, there isn't an issue because if there is a unit available, then the case would be assigned, thus receiving the status of "In Progress". The acceptance criteria is only met when there are no available units in that relevant department. Therefore, the acceptance criteria would need to be

updated that the criteria is only for when there are no available units. For FR-07, after personal testing of the prototype, it has met its expected behaviour since the dashboard is updated without manually refreshing. The suggested GitHub flagged issues because it assumed that automatic updates mean adding a push mechanism (SignalR) to satisfy the stated criteria.

9.2 Representative GitHub Copilot Evidence



Figure 9.2.1: Github Review of FR-02 and FR-07

Figure 9.2.1 shows the prompt used for the requirements review. GitHub Copilot was used to review FR-02, which identified an issue relating to the state management of cases. It discovered that the case can immediately become “In Progress” rather than being “Open” after case creation. It also helped with identifying how the acceptance criteria could be more clear, so that we didn’t have to write the unit as “defect”.

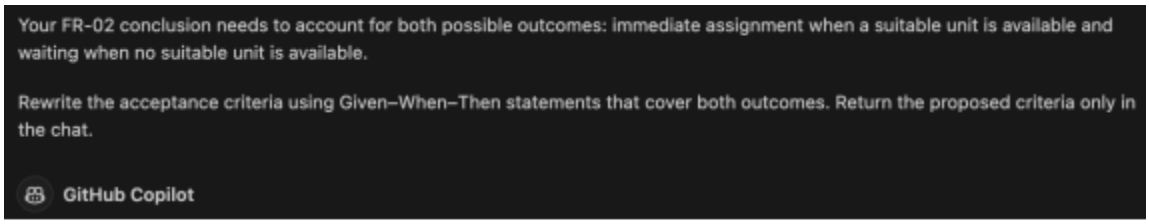


Figure 9.2.2: Follow-up question to clarify Copilot’s assumptions

Figure 9.2.2 shows evidence of questioning Copilot's response instead of agreeing to it blindly. The follow up response told Copilot to account for both immediate assignment, and case queues for FR-02, because it originally accounted for only immediate assignment. Human review is always compulsory, and can help in correcting the LLM's assumptions, and clarifying what the user is looking for.



Figure 9.2.3: Copilot's analysis on Test-isolation

Figure 9.2.3 shows GitHub Copilot analysing why the automated testing passed on one machine, but failed on the other machines. It checked the file-access failures and found a mismatch between the test database and the encryption keys used. It then suggested disabling pooling and using lazy dependency to improve the test isolation. These suggestions were then implemented after review, and the test results passed on all machines.

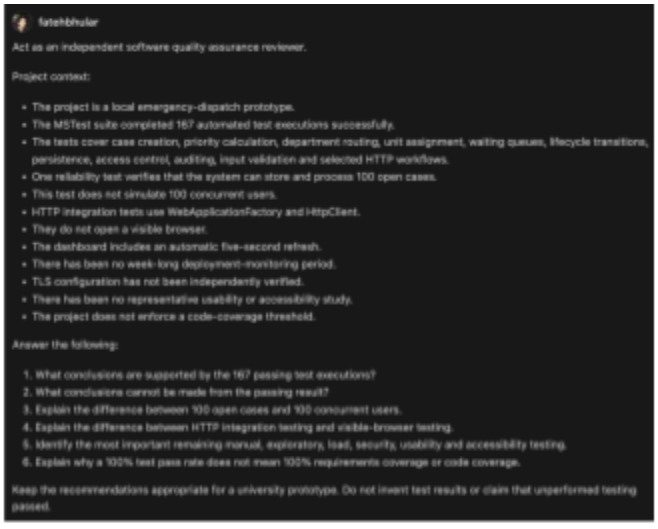


Figure 9.2.4: Copilot's review on test coverage and remaining limitations

Figure 9.2.4 shows evidence of the prompt that instructed Copilot to review what could and could not be concluded for the automated testing results. It correctly explained how the 100-case opening test did not simulate the 100 concurrent users. It also verified that the HTTP integration tests did not replace the visible-browser testing. This indicated that the fully passing test suite did not mean the system is complete or that the system was fully covered in the tests

9.3 Human Review and Validation

GitHub Copilot was not allowed to change any code in our project, without our confirmation. Whenever Copilot was used, we gave it the context of the component we were working on, and any responses were treated like suggestions rather than blindly implementing the changes. Whenever it suggested a change, we checked the requirements, the behaviour it proposed, and the test cases.

FR-02 is one of the requirements that was manually reviewed because Copilot didn't give the correct suggestion. From the prompt in Figure 9.2.1, it only accounted for the immediate case assignment instead of also accounting for the case queue. FR-07 was another requirement that was checked against the requirements because the suggestions from Copilot were not fully correct.

An issue we faced was implementing the local database for permanent persistence (Figure 9.3.1). Copilot suggested test cases that worked on one machine, however it failed on other machines. While AI can suggest code that works, humans are still required to think outside the scope of just working code, and think about how it will work in other environments in order for the system to be deployable.

GitHub Copilot

› Used 1 reference

Database-Testing Problems and Recommendations

Database-Testing Problem	Likely Root Cause	Recommended Correction	Verification Evidence	Remaining Limitation
SQLite temporary-file cleanup failures	SQLite connection pooling retains file handles after tests complete.	Set `Pooling=False` in the SQLite connection string to disable connection pooling.	Confirm that temporary SQLite files are deleted successfully after each test.	Disabling pooling may slightly reduce performance during tests.
Encryption database/key mismatch (`AuthenticationTagMismatchException`)	Database initialized with one encryption key before test dependency replacement.	Use lazy dependency injection to delay database initialization until after the test dependency is replaced.	Confirm that the test database is initialized with the correct encryption key and no mismatch occurs.	Requires careful implementation to ensure all database operations are delayed until after dependency injection.
Early-created database service bypasses test dependency replacement	Database service constructed during `Startup.ConfigureServices` before replacement.	Replace the database dependency in `WebApplicationFactory` before it is resolved.	Confirm that the test database dependency is used instead of the production database.	Requires consistent use of `WebApplicationFactory` for all tests.
Shared database or encryption key across tests	Tests use the same temporary database or encryption key, causing state interference.	Generate unique temporary database and encryption-key paths for each test.	Confirm that each test uses a unique database and encryption key, and no state is shared between tests.	Requires proper cleanup to avoid leaving temporary files on disk.
Tests depend on execution order or shared state	Tests rely on data or state created by other tests.	Ensure each test has isolated setup and teardown, using `[TestInitialize]` and `[TestCleanup]`.	Confirm that tests pass when run in parallel and in random order.	Parallel execution may increase test runtime if setup/teardown is not optimized.

This table summarizes the problems, their causes, and the recommended solutions, along with the evidence needed to verify the corrections and any remaining limitations.

Figure 9.3.1: Copilot solving database testing issue

9.4 AI-Related Quality Risks

AI-related Risk	From this project	How the risk was handled
Incorrect assumptions	The copilot assumed that every new case would get a unit assigned to it, but didn't think about the case queue.	We compared the suggestion against the planned workflow, and clarified FR-02's acceptance criteria.
Overengineering	Copilot suggested using SignalR for real-time communication when we only needed automatic page updates	We stuck with the original plan of the 5-second page refreshing, as using real-time communication was out of scope for this prototype
Limited context	Copilot intentionally did not have direct access to the project planning, so it needed context in the prompts	We supplied Copilot with lots of context not just about the component, but how it would fit into the system workflow. Also checking the suggestions ourselves to make sure it matched the requirements.
Maintainability risk	Copilot suggested fixes may cause bugs in the system and make it harder to debug if not evaluated correctly.	All the suggestions was double checked, and altered by us to ensure we understood the fixes
Confirmation bias	AI is known to be biased towards the user, and doesn't challenge our ideas, even when it isn't a good idea.	We added follow-up prompts giving instructions for Copilot to challenge our ideas to ensure any weak spots in our plan were identified.

The biggest AI-related risk was that Copilot could confidently suggest code or tests that were not suitable for our prototype. This didn't just happen with FR-02 and FR-07 but throughout development and testing. We had to remain proactive and make sure we understood any changes that Coplit suggested. Every response was checked against the requirements table, appendix and test plan.

9.5 Critical Reflection

GitHub Copilot was a productive tool that boosted the overall productivity during the assignment. It helped with the reviewing and testing process by helping with defect organisation, testing limitations, comparing the implemented features with their respective requirements, and system testing. Any technical issues that we faced were explained by Copilot in a way that we understood, making research and development move efficiently.

Our main goal from the last assignment was to be more in control of what we implement in our project. In our last assignment, we were implementing stuff we did not quite fully understand. However, in this assignment we aimed to drive AI and be in control of everything in our project. Any test suggestions, code suggestions, or research was all reviewed by us before letting it into our project.

Overall, Copilot helped improve our understanding of requirements engineering, test isolation, and how to record evidence properly. During development and testing, we used temporary documents that helped summarise our progress. This included what defects we currently had, what updated scope needed to be implemented, and our goals for the next development session. Copilot is a great tool that can boost the development and testing process, but it must also be used carefully. If the user doesn't understand what is being implemented, it will lead to issues down the line, and the later it is found, the more expensive those problems will cost.

10. Quality Improvement and Future Recommendations

10.1 Key Quality Lessons

One of the main lessons taken from this project was that in order for testing to be reliable, the testing environment should be completely isolated from one another. In the case of this project, the issues with connecting and initialising the database affect the test execution, and more prominently, the defect D-05 where HTTP tests could access the developer database. These show that the configuration on how the tests were set up should be checked before any of the results can be trusted. For the future integration tests, they should always be provided separate temporary databases with reliable setup and cleanup in order to protect the production data as well as improving repeatability.

Another lesson was that the scope of the implemented testing should be expanded to include areas such as restarts and failures within specific components. The defect D-01 showed that data was lost whenever the system restarted, while defect D-04 showed how a notification error could make it seem like a completed dispatch has failed. Original testing of only successful workflows was not able to reveal these problems. However github copilot helped suggest additional tests for application restarts and notification failures which was able to identify these defects. This shows that testing should not only be limited to workflows, but also restarts and failures. Github copilot can help suggest missing scenarios and test cases, however these cannot be blindly accepted, and should require human review before being accepted.

Another lesson learnt is that pull requests should use a consistent checklist that covers persistence, permissions, component failures, and isolation. This is because D-01, D-04, and D-05 have shown exactly that these measures need attention, rather than just workflow tests. New databases and HTTP tests should consistently use the isolated temporary database setup as mentioned in section 4.4. and copilot can be used to generate test coverage suggestions, however these cannot be blindly accepted, and should require human review before being accepted.

Lastly, another lesson is that just because every test case passes, that does not necessarily mean that every requirement has been verified. In the context of this project, all 167 automated tests were passed, however there were still some performance, security and usability requirements that still required further testing. Future testing should be more about focusing on these gaps rather than adding more tests. Additionally, manual checks are still required to see if areas such as dashboard refreshing, clear error messages, and that the overall system is easy to use.

10.2 Improvements to the Development and QA Process

An improvement to the QA process that could be done is implementing a review of the requirements, acceptance criteria and planned verification whenever a scope change is agreed. The changes we have made to the scope such as closest based unit assignment and immediate assignment after case creation both affected the implementation and expected test outcomes. Updating these two together would have helped prevent tests from checking outdated behaviour. The RTM should have been kept up to date as soon as there is evidence, such that requirements are marked as partially verified or unverified when tests are not complete.

Additionally, any merges that are done currently undergo a mandatory CI check before any changes are done. This is a problem because a CI pipeline is supposed to ensure quality, and to protect the main branch, ensuring that all the fluids are successful, requiring smoke tests, all suite results to pass, alongside human and code review. The test reports generated should also remain available in order for the failures and defects to be traceable.

10.3 Future Testing and Automation Improvements

The future testing should focus on the unverified and partially verified requirements. The 100 concurrent users requirement is an example of an unfinished test, where instead of 100 users, it was 100 active cases. While the current test confirms the system can maintain 100 open cases, it doesn't prove that the system can handle 100 different users concurrently. Another test that should be prioritised is the 99.9% server uptime test. We didn't have the resources, like a machine that could stay running for longer than 24 hours, so it was not possible during this assignment. These tests will be automated in the future, due to performance testing being important as the application scales.

Security testing should be a big focus in future sprints. Because this is an emergency dispatch priority and coordination system, it is critical that all information is secure, and that there are no issues that prevent the system from running. User details will have stronger encryption, with test cases that are run every time code is deployed. Even though this is not being integrated with a real emergency service, it is

important for sensitive information to remain hidden. When testing the databases, we learned that the test could pass on one machine, but fail on others. This is why in the future, the testing environment should be thought about before stating that the test is complete.

10.4 Future Continuous Quality Engineering

The current implementation of the continuous integration is a valuable inclusion into the development process. However as discussed in **section 5.5**, there are limitations of the current CI. The current quality gate does not not check code coverage and and no vulnerability checking. These are some of the quality gates that could be included into the current CI implementations.

The future implementation could extend the current existing CI pipeline. A proposed improved workflow would begin after the implemented CI towards packaging the application towards a deployable version. This is then staged in the staging environment where additional smoke test and acceptance test could be performed to verify that the version meets the deployment requirements. A manual inspection check would also be needed for the near release version before deploying to production.

Through production of the system, continuous monitoring would be introduced to observe the deployed version, although the current CI integration checks for selected system behavior it does not check when it behaves under operational conditions. These would include measuring response time, availability, errors, database performance and security events.

Continuous deployment could be considered in the future, but until there are sufficient automated quality checks, rollback mechanisms and monitoring procedures it would not be appropriate to introduce it in the current pipeline. Therefore the current continuous quality improvement would be the implementation of continuous delivery.

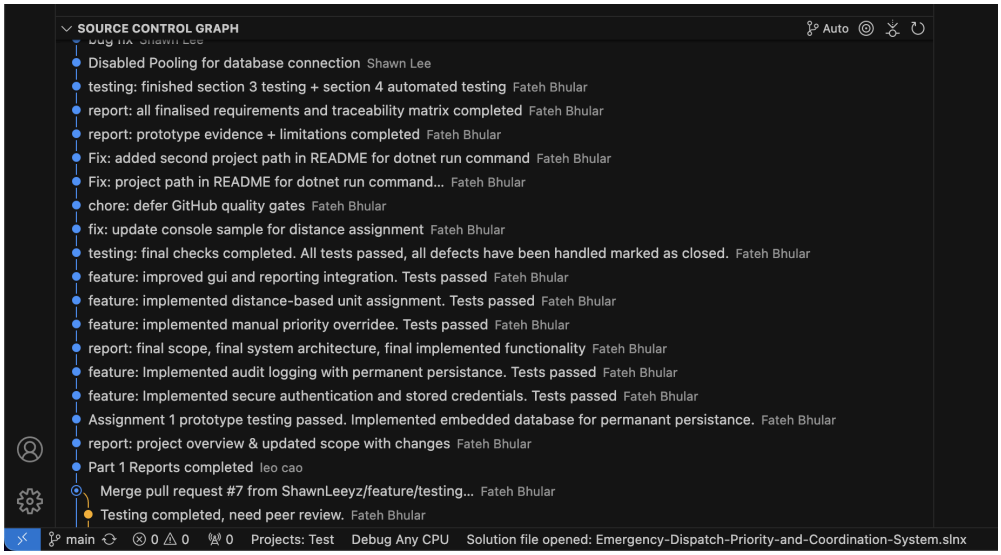
11. Team Contribution and Professional Reflection

11.1 Contribution Statement

Name	Contribution	%
Fateh	- Final Prototype - Section 1: Project Completion - Section 2: Requirements and Traceability - Final prototype Testing and Automated Testing - Section 4: Test Automation and Quality - Section 9: AI-assisted Development & Reflection - Section 10: Development and QA process	33%
Leo	- Section 2: Requirements and Traceability - Section 6: Defect Management and Root Cause Analysis - Section 8: Quality Metrics and Final Release Decision - Section 10: Development and QA process	33%
Shawn	- Section 3: Test execution and final results - Quality Gates and CI pipeline - Section 5: Continuous Integration and DevOps QA - Section 7: Quality Testing - Section 10: Development and QA process	33%

11.2 GitHub and Project Evidence

https://github.com/ShawnLeeyz/Emergency-Dispatch-Priority-and-Coordination-System



11.3 Individual Reflection – Fateh: 23217987

I completed all the major parts in the final prototype, which included database persistence, authentication, priority overriding, distance-based unit allocation, and audit logging. Throughout development, I implemented the core quality assurance practices.

In section 1, I evaluated the differences between the original and final prototypes. Writing it down helped give me a blueprint to what the final prototype should include. In section 2, I had a better understanding of requirements verification and how important it is to link requirements to testing evidence for maintenance. For section 4, I learned how to select tests for automation by using risk . repeatability, and if tests were deterministic, but also that it doesn't cover all testing. Some tests should only be done manually or in other ways because automated testing only helps with repeatable and deterministic outputs. In section 9, I reviewed my AI-assisted development and learned throughout the project that we should drive AI and not let AI drive us. Because LLMs can be wrong, and have different reasoning, we shouldn't trust it blindly, and instead review its outputs and determine for ourselves if the output is good or not. In section 10, I wrote about what future testing will consist of and how we can

improve the testing suite. I learned many lessons about the QA lifecycle and how development should pair coding with testing, operations, and security.

11.4 Individual Reflection – Leo Cao: 23211788

My main contributions were for section 6, 8 and 10. For Section 6, the focus was on defect management and root cause analysis. I learnt that it is important to understand the source of a defect rather than just creating a temporary solution to what first meets the eye as similar defects could reappear. Section 8, included evaluation of final quality metrics and status, evaluating pending risks and checking to see whether or not the product is ready for production. I learnt that achieving 100% test automation does not imply that the system is fully verified.

Section 10 included some lessons learned from our experience throughout the testing and QA project, such as test isolation and failure testing, ensuring requirements stay relevant through changing scope, and improving CI. From this section, I learnt that GitHub Copilot can suggest useful tests, but it would still require human review to see if the outputs are useful and relevant.

11.5 Individual Reflection – Shawn: 23204035

In this project, I have helped with 3 major parts of the project. In section 3, this is where the final test is evaluated: all the tests, including unit, integration, etc. Additionally, from this, I have found issues during testing and documented the debugging process. Furthermore, the test cases are updated to match the new and modified requirements from the first phase of the report. In section 5, I have demonstrated the current development workflow and implemented the continuous integration workflow into the development process. What quality gates were introduced during the first implementation of CI and how CI could be improved. In section 7, the tests were analyzed; instead of focusing on all tests, the test quality was analyzed on the quality areas that are most critical. From this, I learned to select important testing areas that would need more intensive testing evaluation to take into account limitations and possible improvements made in the future, which would satisfy more of the requirements. Throughout this process of test evaluation, I have used GitHub Copilot to determine possible limitations and look through the code and requirements to see whether they match or its suggestions are appropriate for the requirements.

References:

O Aotearoa, R. I. T. R. I. (n.d.). Pressure grew to fix the 111 emergency call system, with the 1958 model still in use. RNZ. https://www.rnz.co.nz/news/emergencies_local/585202/pressure-grows-to-fix-111-emergency-call-system-with-1958-model-still-in-use

1News Reporters. (2026, August 10). St John bracing for another 111 spike amid record demand. 1News. https://www.1news.co.nz/2026/08/10/st-john-bracing-for-another-111-spike-amid-record-demand/

GeeksforGeeks. (2026, May 13). Repository design pattern. GeeksforGeeks. https://www.geeksforgeeks.org/system-design/repository-design-pattern/

GeeksforGeeks. (2026, May 14). Strategy design pattern. GeeksforGeeks. https://www.geeksforgeeks.org/system-design/strategy-pattern-set-1/

Siddique, A. (2026, September 14). *Test automation: Quality engineering foundations* [Lecture slides]. Auckland University of Technology.